

A PROJECT SKILLS LAB REPORT ON
WEATHER-BASED IMAGE AUGMENTATION FOR ENHANCING
OBJECT DETECTION IN AUTONOMOUS DRIVING VISION SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

CSE (ARTIFICIAL INTELLIGENCE)

Under the guidance of

Mr. L. VIJAYA KUMAR,

Assistant Professor



By

H LIKITHA 22751A3120

G HARSHITHA 22751A3116

P KAVYA 22751A3143

SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)

(Approved by AICTE New Delhi & Affiliated to JNTUA, Ananthapuramu)
Chittoor-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

(2025-26)



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

CERTIFICATE

This is to certify that the project report "**WEATHER-BASED IMAGE AUGMENTATION FOR ENHANCING OBJECT DETECTION IN AUTONOMOUS DRIVING VISION SYSTEM**" is a genuine work of

H LIKITHA 22751A3120

G HARSHITHA 22751A3116

P KAVYA 22751A3143

submitted to the department of CSE (Artificial Intelligence), in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY IN CSE (ARTIFICIAL INTELLIGENCE)**, during the academic year 2025-26.

Signature of the SUPERVISOR

Mr. L. VIJAYA KUMAR,

Assistant Professor

Signature of the HEAD OF THE DEPARTMENT

Prof. M. E. PALANIVEL,

Professor & Head of the Department

Submitted for Semester End Examination held on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

Institute Vision

To emerge as a Centre of Excellence for Learning and Research in the domains of engineering, computing and management.

Institute Mission

IM1: Provide congenial academic ambience with state-art of resources for learning and research.

IM2: Ignite the students to acquire self-reliance in the latest technologies.

IM3: Unleash and encourage the innate potential and creativity of students. Inculcate confidence to face and experience new challenges.

IM4: Foster enterprising spirit among students.

IM5: Work collaboratively with technical Institutes / Universities / Industries of National and International repute

Department Vision

To contribute for the society through excellence in Computer Science and Engineering (Artificial Intelligence) with a deep passion for wisdom, culture and values.

Department Mission

M1: Provide congenial academic ambience with necessary infrastructure and learning resources.

M2: Inculcate confidence to face and experience new challenges from industry and society.

M3: Ignite the students to acquire self-reliance in the latest technologies.

M4: Foster Enterprising spirit among students.



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

PROGRAMME EDUCATIONAL OBJECTIVES(PEO's):

PEO1: Expertise with computer science and Engineering, artificial intelligence and machine learning disciplines through quality studies, enabling success in IT industries. **(Professional Competency)**

PEO2: Establish start-up companies or employed in reputed computing industries or government sectors or pursue higher studies in the domain of CSE (AI) **(Successful Career Goals)**

PEO3: Enhance knowledge by updating advanced technological concepts for facing the rapidly changing world and contribute to society through innovation and creativity **(Continuing Education and Contribution to Society).**

PROGRAMME OUTCOMES (PO's):

PO1-Engineeringknowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 - Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 - Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental consideration

PO4 - Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

PO5 - Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6 - The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 – Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8 -Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 - Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10-Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 - Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 - Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)
(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)
CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

PROGRAM SPECIFIC OUTCOMES (PSO's):

PSO1: Demonstrate and analyze the mathematical methodologies to develop computer programs using networking, web design, big data analytics, machine learning algorithms, data science, neural networks and deep learning techniques.

PSO2: Deploy the artificial intelligence techniques to perform human intelligence in creating innovative products and solutions.



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

Course Out Comes for Project Work

On successful completion of the course, students will be able to		POs related to COs
CO1	Demonstrate in-depth knowledge on the selected topic	PO1
CO2	Identify, analyze and formulate complex problem chosen for selected work to attain substantiated conclusions.	PO2
CO3	Design solutions to the chosen selected problem.	PO3
CO4	Under take investigation of selected problem to provide valid conclusions	PO4
CO5	Use the appropriate techniques, resources and modern engineering tools necessary for selected work	PO5
CO6	Apply selected information for sustainable development of the society.	PO6
CO7	Understand the impact of selected concept in the context of environmental sustainability.	PO7
CO8	Understand professional and ethical responsibilities while executing the selected work.	PO8
CO9	Function effectively as individual and a member in the internship.	PO9
CO10	Develop communication skills, both oral and written for preparing and presenting internship report.	PO10
CO11	Demonstrate knowledge and understanding of cost and time analysis required for carrying out the internship.	PO11
CO12	Engage in lifelong learning to improve knowledge and competence in the chosen area of the selected topic.	PO12



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

CO – PO MAPPING

[illegible]



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES,
(Autonomous)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)

CHITTOOR-517127, A.P.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE)

Evaluation Rubrics for Project Work

Rubric(CO)	Excellent(wt=3)	Good(wt=2)	Fair(wt=1)
Selection of Topic (CO1)	Selected a latest topic through complete knowledge of facts and Concepts	Selected a topic through partial knowledge off acts and concepts	Selected a topic through improper knowledge of facts and concepts
Analysis and Synthesis (CO2)	Thorough Comprehension through analysis/ synthesis	Reasonable Comprehension through analysis/ synthesis	Improper Comprehension through analysis/ synthesis
Problem Solving (CO3)	Thorough comprehension about What is proposed in the literature papers	Reasonable comprehension about What is proposed in the literature papers	Improper comprehension about What is proposed in the literature
Literature Survey (CO4)	Extensive literature Survey with standard References	Considerable literature Survey with standard References	Incomplete literature Survey with substandard References
Usage of Techniques & Tools (CO5)	Clearly identified and has complete knowledge Of techniques & tools used in the project work	Identified and has sufficient knowledge of techniques & tools used in the project work	Identified and has in adequate knowledge of techniques & tools used in project work
Project work impact on Society (CO6)	Conclusion of project work has strong impact on society	Conclusion of project work has considerable impact on society	Conclusion of project work has feeble impact on society
Project work impact on Environment (CO7)	Conclusion of project work has strong impact on Environment	Conclusion of project work has considerable impact on environment	Conclusion of project work has feeble impact on environment
Ethical attitude (CO8)	Clearly understands ethical and social practices.	Moderate understanding of ethical and social practices.	Insufficient understanding of ethical and social practices.
Independent earning (CO9)	Did literature survey and selected topic with little Guidance	Did literature survey and selected topic with considerable guidance	Selected a topics suggested by the Supervisor
Oral Presentation (CO10)	Presentation in logical sequence with key points, clear conclusion and excellent language	Presentation with key points, conclusion and good language	Presentation with insufficient key points and improper Conclusion
Report Writing (CO10)	Status report with clear and logical sequence of Chapters using excellent language	Status report with logical sequence of chapters using understandable language	Status report not properly organized
Time and Cost Analysis (CO11)	Comprehensive time and cost analysis	Moderate time and cost analysis	Reasonable time and cost analysis
Continuous learning (CO12)	Highly enthusiastic Towards continuous Learning	Interested in continuous learning	Inadequate interest in continuous learning

ACKNOWLEDGEMENT

A project of this magnitude would not have been possible without the guidance and coordination of many individuals. We consider ourselves fortunate to have received the support and encouragement of such dedicated people, who helped and guided us at every step toward the successful accomplishment of our goal.

Our team expresses its sincere gratitude to the Chairman, Sri **K. Ranganadham Garu**, for his constant encouragement and steadfast support. We also extend our heartfelt thanks to the Managing Trustee, **Mrs. D. A. Kalpaja**, for her continued guidance and developmental support to the institution.

We would like to express our profound gratitude to our Chief Executive, **Prof. K. Nagendra Prasad, Ph.D. (IISc Bangalore)**, for his valuable guidance and encouragement. We also extend our sincere thanks to our Principal, **Dr. N. Venkatachalapathi, M.Tech., Ph.D.**, for his cooperation and valuable suggestions toward the successful completion of our project.

We express our special thanks to our Head of the Department, **Prof. M.E. PALANIVEL.**, for his constant support, valuable suggestions, and guidance toward the successful completion of our project. We also extend our sincere gratitude to our project guide, **Mr. L. VIJAYA KUMAR** for providing us with the opportunity to undertake this work under his esteemed guidance and for his continuous encouragement and support throughout the project.

We also express our sincere gratitude to all the teaching and non-teaching staff of our department for their direct and indirect support throughout our project work. Last but not least, we dedicate this work to our beloved parents and the almighty, who have always been with us, guiding and strengthening us to overcome every challenge.

DECLARATION

We certify that

The work contained in this report is original and has been done by me under the Guidance of my supervisor. The work has not been submitted to any other Institute for any degree or diploma. We have followed the guidelines provided by the Institute in preparing the report. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute. Whenever We have used materials (data, theoretical analysis, figures, and text) from other sources, We have given due credit to them by citing them in the text of the report and giving their details in the references. Further, We have taken permission from the copyright owners of the sources, whenever necessary.

H LIKITHA

22751A3120

G HARSHITHA

22751A316

P KAVYA

22751A3143

ABSTRACT

Autonomous driving systems depend heavily on vision-based perception to understand road environments and make safe decisions. But in poor weather, such as fog, rain, snow, or low light, these systems struggle because visibility drops and images become less clear. Another challenge is the lack of large, weather-specific datasets, which makes it hard for deep learning models to perform well in real-world conditions. To solve these problems, this study introduces an adaptive deep learning perception system combined with a weather-based image augmentation framework for autonomous vehicles. The system uses EfficientNetB3 to automatically classify weather conditions, such as rain, foggy, or snowy scenes. Weather-based image augmentation increases data diversity and helps the model learn features that work across different weather types, making it more robust. For real-time object detection, the system uses YOLOv8, which can quickly and accurately identify cars, pedestrians, and other road objects with low computing delay. By combining object detection and weather classification in one flexible system, it can adjust to changing conditions without needing complex image restoration. Tests show that this approach improves detection accuracy, stability, and reliability compared to standard vision-based methods, making it efficient, scalable, and ready for real-world use in autonomous driving.

LIST OF FIGURES

FIG NO	FIG NAME	PAGE NO
5.3	System Architecture	20
6.4.1	Use case Diagram	26
6.4.2	Class Diagram	27
6.4.3	Activity Diagram	28
6.4.4	Sequence Diagram	29
6.4.5	Deployment Diagram	30
B.a	User Interface	47
B.b	Uploading Image	48
B.c	Detected weather & class confidences	49
B.d	Object Detection on Image	50
B.e	Invalid Image Upload	51
B.f	Confusion Matrix	52

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
ACDC	Adverse Conditions Dataset with Correspondences
API	Application Programming Interface
BDD100K	Berkeley DeepDrive 100K Dataset
BGR	Blue Green Red
CLI	Command Line Interface
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DAWN	Detection in Adverse Weather Nature
Flask	Flask (Python Micro Web Framework)
GB	Gigabyte
GPU	Graphics Processing Unit
GTX	Giga Texel Shader eXtreme (NVIDIA GPU series)
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
NVIDIA	Nvidia Visual Computing (brand)
ONNX	Open Neural Network Exchange
OS	Operating System
PNG	Portable Network Graphics
PO	Program Outcome
RAM	Random Access Memory
RGB	Red Green Blue
T4	Tesla T4 (Google Colab GPU)
TC	Test Case
TTA	Test-Time Augmentation
UI	User Interface
UML	Unified Modeling Language
WEDGE	Weather-Diverse Dataset for Generalization Evaluation
YOLO	You Only Look Once

INDEX

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF ABBREVIATIONS	iii
1	INTRODUCTION	1-4
	1.1 MOTIVATION	2
	1.2 OBJECTIVE	2
	1.3 PROBLEM STATEMENT	3
2	LITERATURE SURVEY	5-6
3	SYSTEM ANALYSIS	7-14
	3.1 EXISTING SYSTEM	7
	3.1.1 DEMERITS	8
	3.2 PROPOSED SYSTEM	8
	3.2.1 MERITS	10
	3.3 FEASIBILITY STUDY	10
	3.3.1 ECONOMICAL FEASIBILITY	10
	3.3.2 TECHNICAL FEASIBILITY	11
	3.3.3 SOCIAL FEASIBILITY	11
	3.4 SYSTEM SPECIFICATIONS	11
	3.4.1 FUNCTIONAL REQUIREMENTS	11
	3.4.2 NON FUNCTIONAL REQUIREMENTS	12
	3.4.3 SOFTWARE AND HARDWARE REQUIREMENTS	13
4	SYSTEM DEVELOPMENT MODEL	15-17
	4.2 AGILE MODEL	15
	4.3 PHASES OF SYSTEM DEVELOPMENT	16
5	SYSTEM DESCRIPTION	18-21
	5.1 PROBLEM IDENTIFICATION	18
	5.2 OVERVIEW OF THE SYSTEM	19
	5.3 SYSTEM ARCHITECTURE & DESCRIPTION	19
6	SYSTEM DESIGN	22-30
	6.1 MODULE AND ITS DESCRIPTION	22
	6.1 INPUT DESIGN	25

	6.2 OUTPUT DESIGN	25
	6.4 UML DIAGRAMS	26
	6.4.1 USE CASE DIAGRAM	16
	6.4.2 CLASS DIAGRAM	27
	6.4.3 ACTIVITY DIAGRAM	27
	6.4.4 SEQUENCE DIAGRAM	28
	6.4.5 DEPLOYMENT DIAGRAM	29
5	SYSTEM TESTING	31-34
	5.1 TYPES OF TESTING	31
	5.1.1 UNIT TESTING	31
	5.1.2 INTEGRATION TESTING	32
	5.1.3 FUNCTIONAL TESTING	32
	5.1.4 SYSTEM TESTING	32
	5.1.5 WHITE BOX TESTING	33
	5.1.6 BLACK BOX TESTING	33
	5.2 TEST CASES	34
6	SYSTEM IMPLEMENTATIONS	35-38
	CONCLUSION AND FUTURE ENHANCEMENT	39
	REFERENCES	41
	APPENDIX	42-52
	A SOURCE CODE	42
	B SCREENSHOTS	48
	ANNEXURE-I	54

CHAPTER 1

INTRODUCTION

Computer vision is essential for autonomous driving systems to understand and respond to intricate real-world situations. Important operations like object detection, lane recognition, and obstacle avoidance are handled by deep learning models that have been trained on massive image sets. However, the majority of public datasets only contain photos shot in favorable weather and lighting conditions. Under actuality, autonomous cars must function dependably under a variety of difficult circumstances, including poor light, rain, fog, snow, and sandstorms.

Autonomous driving vision systems are seriously at risk from weather-induced visual deterioration. Large-scale real-world driving data collection and annotation across all potential weather situations is costly, time-consuming, and often impractical. Therefore, when exposed to unforeseen or harsh weather circumstances, vision models trained on small or biased datasets typically perform badly.

A workable and scalable solution to this issue is weather-based image augmentation. Augmentation techniques enable object detection and classification models to be trained on a variety of environmental variations without requiring substantial real-world data collection by artificially converting clear-weather images into versions degraded by rain, fog, snow, or haze while maintaining semantic content.

A weather-aware adaptive vision pipeline for autonomous driving is proposed and implemented in this project. For each of the four weather categories—fog, rain, snow, and sand—the system applies class-specific augmentation using the DAWN dataset, BDD100K dataset(rain), ACDC dataset(rain,fog). YOLOv8 is used for real-time object identification, and EfficientNet uses two-phase transfer learning for weather classification. A Flask-based web application that incorporates both models takes pictures of road scenes and returns outcomes of object detection and weather classification at the same time. The system is made to be effective, scalable, and deployable in real-world autonomous driving situations.

1.1 MOTIVATION

Vision-based perception technologies are becoming increasingly critical to the safe and reliable operation of autonomous vehicles. These systems primarily rely on real-time object detection models that identify traffic signs, cars, pedestrians, and obstacles. Even while deep learning-based object detection has advanced significantly, poor weather conditions like rain, fog, snow, and low light still cause problems for existing vision systems. Since autonomous vehicles are expected to operate dependably across a range of environments, this constraint presents a significant risk to public safety.

One of the primary sources of this problem is the imbalance in training datasets, which are mostly collected under perfect, clear weather conditions. Obtaining large-scale, well-annotated datasets covering all potential weather variations is costly, logistically challenging, and time-consuming. It is extremely difficult to reliably capture uncommon and extreme weather occurrences, which results in inadequate data representation during model training and poor generalization to unseen or degraded visual settings.

Weather-based image enhancement is a practical and cost-effective approach. By artificially simulating adverse weather effects on pre-existing datasets, data diversity can be expanded without requiring new collection activities. Augmentation processes that are exclusive to each weather class, as opposed to being performed randomly, enable models to learn distinct visual characteristics of each weather state, improving detection robustness and reliability in real-world circumstances. In addition to technological ones, there is an urgent need for more weather-resistant autonomous driving systems from a practical and societal perspective.

1.2 OBJECTIVE

The primary goal of this project is to apply weather-based image augmentation techniques to improve the performance and robustness of weather classification and object detection models in autonomous driving vision systems under diverse adverse weather conditions. The specific objectives are as follows:

1. To investigate and evaluate the impact of adverse weather conditions such as fog, rain, snow, and sand on vision-based object detection and classification systems used in autonomous driving.
2. To design and implement a class-specific weather augmentation pipeline using the Albumentations library, where each weather category receives augmentation that reflects its unique visual characteristics, avoiding cross-class confusion.
3. To train an EfficientNetB0-based weather classification model using two-phase transfer learning, with proper preprocessing using EfficientNet's native `preprocess_input` function, label smoothing, class weighting, and training callbacks.
4. To apply Test-Time Augmentation (TTA) at inference time, averaging predictions over multiple augmented passes to improve classification accuracy and robustness on out-of-distribution images.
5. To integrate YOLOv8 for real-time object detection on road scene images, enabling the system to identify vehicles, pedestrians, and traffic signs simultaneously with weather classification.
6. To deploy the complete pipeline as a Flask web application with an HTML interface, allowing users to upload images and receive combined weather classification and object detection results.
7. To demonstrate an economical and scalable approach to improving autonomous driving perception systems without the need for large-scale real-world data collection under severe weather conditions.

1.3 PROBLEM STATEMENT

Deep learning-based image classification and object-level perception are the core components of autonomous driving vision systems that enable safe navigation and decision-making. However, these systems perform substantially worse in adverse weather circumstances including rain, fog, snow, and sandstorms because to diminished vision, background noise, and the loss of crucial visual components. Because most of the current vision models were trained on datasets mostly composed of

clear-weather images, they have limited robustness and poor generalization when employed in real-world driving situations with various environmental conditions.

One major practical difficulty is the absence of large-scale annotated datasets that reliably record all weather changes. In challenging or severe environments, gathering and classifying real-world data is costly, time-consuming, and often dangerous. Additionally, a snow augmentation may inadvertently be applied to a rain image or vice versa because typical augmentation pipelines often use augmentation techniques without class specificity. Decreased classification accuracy may result from this cross-class augmentation's introduction of label noise and confusion of the model.

Preprocessing mismatches are another technical issue. EfficientNetB0 anticipates that its internal `preprocess_input` function, which normalizes inputs based on the statistics of the ImageNet training distribution, will process the input pixel values. Bypassing this internal preprocessing and using the more straightforward `rescale=1/255` normalization instead, model accuracy is significantly reduced—sometimes by more than 20 percentage points. This error is fixed in this project, however it occurs in many implementations.

Furthermore, models are frequently stored at the last training epoch rather than the epoch attaining the best validation accuracy in the absence of appropriate training callbacks like `ModelCheckpoint`, `EarlyStopping`, and `ReduceLROnPlateau`, resulting in less-than-ideal deployments. Through a rectified, class-specific augmentation pipeline, appropriate EfficientNet preprocessing, TTA at inference, and a full dual-model deployment that combines YOLOv8 object recognition with weather classification, this research tackles all of these issues.

CHAPTER 2

LITERATURE SURVEY

Vision-based perception systems are necessary for intelligent transportation systems, particularly for smart mobility and autonomous driving applications. However, their efficacy significantly deteriorates under adverse weather conditions such as rain, fog, and snow. To improve system reliability, researchers have investigated weather-aware data augmentation, robust deep learning architectures, and the creation of synthetic datasets.

Z. Wang et al. (2025) introduced a deep learning-based parking space recognition framework specifically designed to address adverse weather conditions. Their work presents a simulated weather augmentation technique that artificially adds rain, snow, and fog effects to actual parking images in order to increase dataset diversity. Experiments using CNN architectures such as ResNet18 and DenseNet121 demonstrated that models trained with weather augmentation outperform those trained only on single-weather data. Importantly, the study identified a critical fog density threshold beyond which detection performance drops sharply, highlighting the sensitivity of CNN-based systems to visual degradation.

Marathe et al. (2023) proposed WEDGE, a multi-weather autonomous driving dataset created with the assistance of vision-language models. This dataset includes synthetic images depicting diverse weather conditions alongside bounding box annotations for object detection tasks. The contribution addresses the shortage of labeled adverse-weather datasets and enables benchmarking of perception models under challenging atmospheric conditions. However, the primary focus of WEDGE is dataset creation rather than system-level deployment or the development of application-specific pipelines such as those used for parking or weather-adaptive detection.

Bayramov et al. (2025) presented weather-informed vision enhancement techniques designed to improve object detection accuracy under challenging conditions. Their approach involves preprocessing input images using weather-aware augmentation modules before passing them to

conventional object detectors. Experimental results show that improving visibility in degraded weather conditions leads to better detection performance. However, this method functions as a preprocessing step rather than integrating augmentation directly into the training pipeline, adding computational complexity and making it less suitable for real-time edge deployment.

Aloufi et al. (2024) proposed a multi-objective framework that integrates weather classification and object detection into a single deep learning model. The system explicitly identifies the current weather condition and adapts its detection strategy accordingly. Experiments on adverse-weather datasets demonstrated improved robustness compared to conventional single-task detectors. Despite its effectiveness, the added complexity of a joint model may limit its practical applicability in resource-constrained environments such as embedded hardware on autonomous vehicles.

Gurbindo et al. (2025) explored the use of diffusion-based generative models, specifically Instruct Pix2Pix, to produce realistic weather-augmented images for autonomous driving datasets. Their technique improves object detector performance under unseen adverse weather using YOLO and Faster R-CNN architectures. While diffusion-based augmentation produces high-quality synthetic images, the computational cost of generating data at scale makes this approach impractical for rapid iteration or large-scale training pipelines.

Drawing from these findings, the present project addresses several gaps. Rather than using a unified or random augmentation strategy, this work implements class-specific augmentation for each weather category (fog, rain, snow, sand) to preserve visual distinctiveness. Rather than relying on image restoration preprocessing, augmented images are fed directly into the training pipeline, avoiding additional latency. The EfficientNet classifier is trained using the correct internal preprocessing function rather than a generic rescale, and TTA is used at inference to improve out-of-distribution performance. Finally, the integration of YOLOv8 for simultaneous object detection in a deployable Flask web application represents a practical advancement over purely experimental or dataset-centric prior works.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Deep learning-based perception models trained on datasets gathered in clear or normal weather conditions provide the basis of contemporary autonomous vision and intelligent mobility technologies. In unfavorable weather, such as fog, rain, snow, and poor light, these devices operate far worse than they do in favorable weather. The accuracy of feature extraction and object recognition is significantly impacted by reduced visibility, contrast loss, motion blur, and background noise.

Some current methods use image restoration techniques like dehazing, deraining, and desnowing as preprocessing processes prior to object detection in order to overcome the lack of real-world unfavorable weather data. Nevertheless, these approaches increase processing complexity and could exclude crucial visual elements needed for precise edge-based and real-time applications.

Recently, simulation-based weather augmentation techniques have been studied to give larger training datasets. These methods entail enhancing clear images with synthetic fog, rain, or snow effects. Specifically, binary classification tasks like parking spot occupancy detection are the focus of current parking management systems. In order to improve robustness under particular weather circumstances and do away with explicit picture restoration, these systems employ convolutional neural networks, such as ResNet and DenseNet, that are trained directly on weather-degraded photos.

The scope and utility of current technologies are constrained despite these developments. They focus on classification tasks rather than real-time object recognition, are frequently built for offline analysis, and assume known environmental conditions. Additionally, they don't have the adaptive sensing techniques and explicit weather awareness needed for dynamic autonomous driving.

3.1.1 DEMERITS

Current systems do not adapt effectively to unfavorable weather circumstances including fog, rain, snow, and low light levels because they were mostly trained on clear-weather datasets.

In real-world situations, diminished visibility, contrast loss, and ambient noise cause significant performance reduction.

Model robustness is limited by the scarcity, cost, and difficulty of annotating real-world severe weather datasets.

Many methods rely on picture restoration techniques, such as deraining or dehazing, which are inappropriate for real-time applications and increase processing complexity.

Inaccurate detection findings may occur from the removal of important visual characteristics by image restoration techniques.

Instead of focusing on comprehensive object detection, existing systems frequently concentrate on specific objectives like binary classification (such as parking occupancy).

Models cannot dynamically adjust to changing environmental conditions if they lack explicit weather knowledge.

Most systems are designed for offline analysis and are not optimized for deployment in real-time autonomous driving environments.

3.2 PROPOSED SYSTEM

The proposed system is an intelligent adverse weather classification and object detection framework designed specifically for autonomous driving applications in harsh environmental conditions. Unlike conventional systems that operate under clear weather assumptions, the proposed system integrates a dual-task deep learning architecture capable of simultaneously classifying weather conditions and detecting road-relevant objects within a single inference pipeline.

The core of the proposed system is built upon EfficientNetB3, a highly efficient convolutional neural network pretrained on ImageNet and fine-tuned on a curated multi-source adverse weather dataset. The backbone is extended with a dual-output head: a primary weather classification head that outputs

probability scores across four adverse weather categories — fog, rain, sand, and snow — and a secondary validity detection head that determines whether the input image constitutes a valid road or driving scene. This dual-head architecture allows the system to simultaneously classify the prevailing weather condition and filter out irrelevant or invalid inputs in a single forward pass.

The training dataset is constructed by combining three complementary sources. The DAWN dataset provides the foundational adverse weather driving images across fog, rain, sand, and snow categories. The ACDC dataset contributes 1,000 real sensor-captured foggy driving images and 1,000 real rainy driving images, significantly improving the model's exposure to authentic degradation patterns. The BDD100K dataset further supplements the rain category with 500 additional real-world rainy driving scenes. Class-specific augmentation strategies are applied to each weather category to amplify dataset diversity while preserving the visual characteristics unique to each condition. Fog augmentation emphasises uniform brightness elevation and soft Gaussian blur to simulate haze diffusion. Rain augmentation applies directional motion blur, blue-channel shifting, and reduced brightness to replicate wet-road reflectance and rain streak physics. Snow and sand augmentations simulate their characteristic brightness and colour temperature shifts respectively.

Parallel to the weather classification component, the system deploys a YOLOv8-small model for real-time object detection. The detector is configured with weather-adaptive confidence thresholds: lower thresholds are applied under fog and sand conditions to compensate for reduced visibility and object contrast, while higher thresholds are used under rain and snow where false positives are more likely. This weather-conditioned detection strategy ensures that detection sensitivity is dynamically adjusted to the prevailing environmental degradation level.

The complete system is deployed as a Flask web application providing a browser-based interface for real-time adverse weather classification and object detection. Users upload a driving scene image through the interface, and the system returns the predicted weather condition with confidence score, per-class probability distribution, a list of detected road objects, and the annotated detection output image with bounding boxes rendered by YOLOv8.

3.2.1 MERITS

Applies class-specific augmentation strategies for each weather category, preserving visual distinctiveness and eliminating cross-class label confusion.

Uses EfficientNet's native preprocess_input function, correcting a critical preprocessing error that causes up to 20-30% accuracy loss in standard implementations.

Employs two-phase transfer learning with EfficientNetB0, enabling efficient training with limited weather-specific data.

Applies Test-Time Augmentation at inference, improving classification accuracy and robustness on unseen or out-of-distribution images.

Integrates YOLOv8 for real-time road object detection with low computational latency.

Uses ModelCheckpoint, EarlyStopping, and ReduceLROnPlateau callbacks to ensure the best model is saved and training does not overfit.

Applies class-weighted loss to handle the inherent class imbalance in the DAWN dataset.

Delivers a complete, deployable Flask web application combining weather classification and object detection in a single pipeline.

Eliminates the need for computationally expensive image restoration preprocessing, reducing latency and enabling real-time operation.

3.3 FEASIBILITY STUDY

A feasibility study determines if a suggested system can be successfully developed and implemented within the constraints of time, money, resources, and technology. This section assesses the project's viability from an economic, technological, and social standpoint.

3.3.1 Economical Feasibility

The project's main dependence on open-source tools and publicly accessible datasets makes it financially feasible. The Kaggle API provides a free download for the DAWN dataset. Open-source and publicly available deep learning frameworks include TensorFlow, Keras, and Ultralytics YOLOv8. Additionally, the augmentation library Albumentations is open-source. Google Colab, which offers free access to GPU resources (T4 GPU), is used for model training, doing away with the

requirement for pricey dedicated hardware. The project's low total development costs attest to its viability from an economic standpoint.

3.3.2 Technical Feasibility

The project is technically feasible because it builds entirely on well-established deep learning and computer vision technologies. EfficientNet, made available through the TensorFlow/Keras ecosystem, is a proven architecture for image classification tasks. Albumentations is a widely adopted library for high-performance image augmentation. YOLOv8, provided by Ultralytics, is state-of-the-art for real-time object detection and is actively maintained with extensive documentation. Flask is a lightweight and well-supported Python web framework suitable for model serving. All training is performed in Google Colab with GPU support, and all inference is tested on CPU hardware to ensure portability. The implementation uses Python 3.8+, and all dependencies are installable via pip. The project is technically feasible given these mature, accessible tools.

3.3.3 Social Feasibility

The proposed system is both socially feasible and beneficial. By improving the reliability of autonomous driving perception in bad weather, the technique immediately contributes to road safety, a subject of broad public concern. More precise object detection in fog, rain, and snow can increase public confidence in autonomous driving technology and reduce collisions caused by low visibility. Because of its Flask-based web interface, end users do not require technical expertise to present and evaluate the system. The project also promotes future research in the area of trustworthy autonomous driving perception by addressing recognized shortcomings in existing weather augmentation pipelines.

3.4 SYSTEM SPECIFICATIONS

The system specifications define the functional and non-functional requirements of the proposed system.

3.4.1 Functional Requirements

Dataset Management: The system must download and organize the DAWN dataset from Kaggle, perform an 80/20 train-validation split, and apply class-specific augmentation per weather category.

Weather-Based Image Augmentation: The system must apply fog, rain, snow, and sand augmentation using Albumentations, with distinct augmentation parameters per class to preserve visual separability.

Weather Classification: The system must train an EfficientNetB0 classifier using two-phase transfer learning, correct EfficientNet preprocessing, label smoothing, class weighting, and training callbacks.

Test-Time Augmentation: The system must apply TTA at inference by averaging predictions over ten augmented passes.

Object Detection: The system must run YOLOv8n on the uploaded image, draw bounding boxes around detected road objects, and return detection labels.

Web Interface: The system must provide a Flask-based web application with an HTML frontend allowing image upload and display of weather classification and object detection results.

Model Persistence: The system must save the best model checkpoint and load it explicitly before inference.

3.4.2 Non-Functional Requirements

Performance: The system must produce classification and detection results within a reasonable latency, leveraging GPU acceleration during training and operating on CPU during web inference.

Reliability: The system must produce consistent results for the same input image across multiple calls.

Maintainability: The system must be modular, with separate scripts for training and inference/deployment, enabling independent updates.

Scalability: The system must support the addition of new weather categories, larger datasets, or upgraded model architectures without major structural changes.

Usability: The system must be operable through a browser-based interface without requiring end-user knowledge of deep learning or Python.

3.4.3 Software and Hardware requirements

Software requirements:

Python 3.8+

The fundamental programming language used throughout the system's implementation is Python. It is well suited for computer vision and machine learning applications due to its wide ecosystem and simple syntax. The language facilitates quick development and deployment and allows for the seamless integration of several libraries.

Tensorflow & Keras

The two main deep learning frameworks used to construct and train neural networks are TensorFlow and Keras. While Keras's modular and intuitive interface makes model development easier, TensorFlow offers a highly optimized computational backend with support for GPU acceleration. When combined, they simplify the development, training, and assessment of deep learning models.

EfficientNet

Advanced picture augmentation is carried out using albumentations to improve dataset variety. In order to train models for real-world driving scenarios, it makes it possible to simulate difficult weather conditions including fog, rain, snow, and dust. This enhances robustness and generalization.

OpenCV

OpenCV is incorporated for handling image preprocessing tasks. It supports efficient image loading, resizing, color space transformations, and augmentation operations. The library ensures that all input data is consistently formatted before being passed to the learning models.

YOLOv8

For real-time object detection in road situations, YOLOv8 is used. It is ideal for situations where prompt decision-making is essential because of its high-speed inference and precise detection capabilities. Under different weather situations, the model recognizes items like cars, pedestrians, and obstructions.

Flask

The system's backend is developed using Flask. It makes it easier to create web-based interfaces and APIs that let people engage with the models that have been trained. Flask's lightweight design guarantees effective request processing and simple deployment.

Tool / Library	Purpose
Python 3.8+	Primary programming language for the entire implementation
TensorFlow / Keras	Deep learning framework used for EfficientNet training
EfficientNet (ImageNet)	Pre-trained backbone for weather classification with transfer learning
Albumentations	Weather-specific image augmentation (fog, rain, snow, sand)
OpenCV (cv2)	Image reading, color conversion, and saving augmented images
Ultralytics YOLOv8	Real-time object detection on road scene images
Flask	Backend web framework to serve the application
NumPy	Numerical computation and array manipulation
scikit-learn	Computing class weights for handling class imbalance
Matplotlib	Visualization of predictions and training history
Google Colab	Cloud-based training environment with free GPU (T4) access
Kaggle API	Dataset download (DAWN dataset)

Hardware Requirements:

Component	Specification
Processor	Intel Core i5 or equivalent (minimum); Intel Core i7 or higher (recommended)
RAM	8 GB minimum; 16 GB or more recommended
GPU	NVIDIA CUDA-enabled GPU (e.g., GTX 1060 or higher); Google Colab T4 GPU (cloud-based)
Storage	50 GB or more of free disk space
Operating System	Windows 10 / Ubuntu 20.04 or higher / macOS (64-bit)
Internet Connection	Required for dataset download, Kaggle API access, and Google

CHAPTER 4

SYSTEM DEVELOPMENT MODEL

An incremental and iterative development approach was used to create the Weather-Based Image Augmentation for Improving Object Detection in Autonomous Driving Vision System. This strategy was chosen since the system incorporates several interrelated elements, each of which needs separate development, validation, and gradual integration. These elements include dataset preparation, augmentation pipeline, deep learning model training, real-time detection, and online deployment.

4.1 AGILE MODEL

Agile is an iterative and incremental approach where the system is developed in small cycles called sprints, with continuous testing, feedback, and improvement at every stage rather than completing the entire system at once.

In this project, the Agile model is reflected through the step-by-step development of seven independent modules — dataset preparation, augmentation, preprocessing, model training, inference, object detection, and web deployment. Each module was built, tested, and refined before moving forward, which aligns with Agile's principle of iterative delivery.

The adaptive nature of Agile is clearly seen when a critical preprocessing error was identified mid-development and corrected immediately, replacing `rescale=1/255` with EfficientNet's native `preprocess_input` function, recovering up to 20–30% accuracy loss. Similarly, the two-phase training of EfficientNet and the addition of Test-Time Augmentation represent continuous improvement cycles typical of Agile development.

In this project, the development began with dataset preparation, followed by the augmentation pipeline, then model training, inference, object detection, and finally web deployment. Each stage was tested and validated before moving to the next, ensuring that errors were caught early and corrected progressively.

The iterative nature is evident in the two-phase training of EfficientNet, where the model is first trained with frozen layers and then fine-tuned in a second pass. Similarly, the correction of the

preprocessing function and the addition of Test-Time Augmentation reflect iterative refinement of the pipeline based on observed results.

- Each module to be developed and tested independently before integration.
- Early detection of accuracy bottlenecks (e.g., the EfficientNet preprocessing error causing 20–30% accuracy loss).
- Progressive improvement through two-phase training and the addition of Test-Time Augmentation (TTA).
- A deployable intermediate product (Flask app) available even while individual modules are being refined.

4.2 PHASES OF DEVELOPMENT

Phase 1 : Requirements and Feasibility Analysis

The team analyzed the limitations of existing autonomous driving vision systems, particularly their poor performance under adverse weather. The DAWN dataset was identified as a publicly available and suitable resource. Feasibility was assessed across economic (open-source tools, free GPU via Google Colab), technical (mature libraries: TensorFlow, YOLOv8, Albumentations, Flask), and social dimensions (road safety impact). Functional and non-functional requirements were clearly defined.

Phase 2 : System Design

The system was designed as a modular pipeline of seven interconnected modules:

- Dataset Collection and Organisation
- Class-Specific Augmentation
- Data Generator and Preprocessing
- Model Architecture and Training (EfficientNet)
- Inference with Test-Time Augmentation
- Object Detection with YOLOv8
- Flask Web Application and Frontend

UML diagrams were produced to document the architecture and data flow.

Phase 3: Iterative Implementation

Development proceeded module by module:

- The dataset was downloaded, split (80/20), and organized.
- Class-specific augmentation pipelines were built and validated per weather category.
- EfficientNetB0 was trained in two phases: first with a frozen base (Phase 1, 30 epochs, lr=1e-3), then with fine-tuning of the top 100 layers (Phase 2, 20 epochs, lr=1e-4).
- A critical preprocessing fix was applied — replacing `rescale=1/255` with EfficientNet's native `preprocess_input` — resolving a 20–30% accuracy loss present in many baseline implementations.
- Callbacks (ModelCheckpoint, EarlyStopping, ReduceLROnPlateau) were integrated to ensure the best model checkpoint was always preserved.
- TTA was implemented at inference for more robust predictions.
- YOLOv8n was integrated for real-time object detection.

Phase 4: Integration and Testing

All modules were integrated and tested end-to-end. Testing covered unit testing (per module), integration testing (data flow between modules), system testing (full pipeline), performance testing (accuracy, precision, recall, mAP), white box testing (internal logic), and black box testing (observable inputs/outputs via the web interface).

Phase 5 : Deployment

The complete pipeline was deployed as a Flask web application (`app.py`). Users can upload road scene images via a browser interface and receive simultaneous weather classification and object detection results, with annotated bounding box outputs rendered in the HTML frontend.

CHAPTER 5

SYSTEM DESCRIPTION

5.1 PROBLEM IDENTIFICATION

Autonomous driving vision systems depend heavily on camera-based perception to detect objects, recognize lanes, and make real-time navigation decisions. However, these systems face serious performance degradation under adverse weather conditions such as fog, rain, snow, and sandstorms. The core problems identified in this project are as follows.

Most existing deep learning models for autonomous driving are trained on datasets collected under clear and favorable weather conditions. As a result, these models fail to generalize effectively when exposed to real-world weather variations, leading to reduced detection accuracy and unreliable predictions.

Collecting and annotating large-scale real-world datasets that cover all possible weather conditions is extremely costly, time-consuming, and often dangerous. This creates a significant data scarcity problem, especially for rare or extreme weather scenarios.

Existing augmentation pipelines commonly apply weather effects randomly across all image classes without considering class-specific visual characteristics. This cross-class augmentation introduces label noise and confuses the model, reducing overall classification accuracy.

A critical technical issue was also identified in many standard implementations where EfficientNetB0's internal preprocessing is bypassed by using a generic `rescale=1/255` normalization instead of the native `preprocess_input` function. This mismatch causes a significant accuracy loss of up to 20–30 percentage points.

Furthermore, without proper training callbacks such as `ModelCheckpoint`, `EarlyStopping`, and `ReduceLROnPlateau`, models are often saved at the last training epoch rather than the epoch achieving the best validation accuracy, resulting in suboptimal deployments.

5.2 OVERVIEW OF THE SYSTEM

The proposed system is a Weather-Aware Adaptive Vision Pipeline designed to improve the robustness and reliability of autonomous driving perception under diverse adverse weather conditions. It combines weather classification and real-time object detection into a single unified and deployable framework.

The system uses the publicly available DAWN dataset, which contains road scene images categorized into four weather classes — fog, rain, snow, and sand. A class-specific augmentation pipeline built using the Albumentations library applies unique weather effects to each category, preserving visual distinctiveness and eliminating cross-class confusion.

For weather classification, EfficientNet pre-trained on ImageNet is used as the backbone model. It is trained using a two-phase transfer learning strategy — first training only the custom classification head with the base model frozen, and then fine-tuning the top layers of the base model at a lower learning rate. EfficientNet's native `preprocess_input` function is used for correct normalization.

At inference time, Test-Time Augmentation (TTA) is applied by averaging predictions over ten augmented passes of each input image, improving classification stability and robustness on unseen images.

For real-time object detection, YOLOv8n is integrated to identify road objects such as vehicles, pedestrians, and traffic signs simultaneously with weather classification. The entire system is deployed as a Flask web application with an HTML frontend, allowing users to upload road scene images and receive both weather classification and object detection results through a simple browser interface.

5.3 SYSTEM ARCHITECTURE AND DESCRIPTION

The system is designed as a modular pipeline consisting of seven interconnected components. The architecture ensures that each stage of the workflow is independently maintainable and collectively forms a complete end-to-end perception system.

Dataset Collection and Organisation The DAWN dataset is downloaded using the Kaggle API and organized into four weather categories — fog, rain, snow, and sand. An 80/20 train-validation split is performed at the image level to prevent data leakage between training and validation sets.

Class-Specific Augmentation Four separate augmentation pipelines are defined using Albumentations, one for each weather class. Fog images receive layered blur and fog effects. Rain images receive directional streaks with saturation changes. Snow images receive controlled brightness boosting. Sand images receive RGB channel shifts, Gaussian noise, and contrast reduction. Augmented images are merged with original training images to form the final dataset.

Data Generator and Preprocessing Keras ImageDataGenerators are created for training and validation using EfficientNet's native preprocess_input function. The training generator applies additional runtime augmentations such as rotation, zoom, and horizontal flipping. Class weights are computed using scikit-learn to handle class imbalance in the dataset.

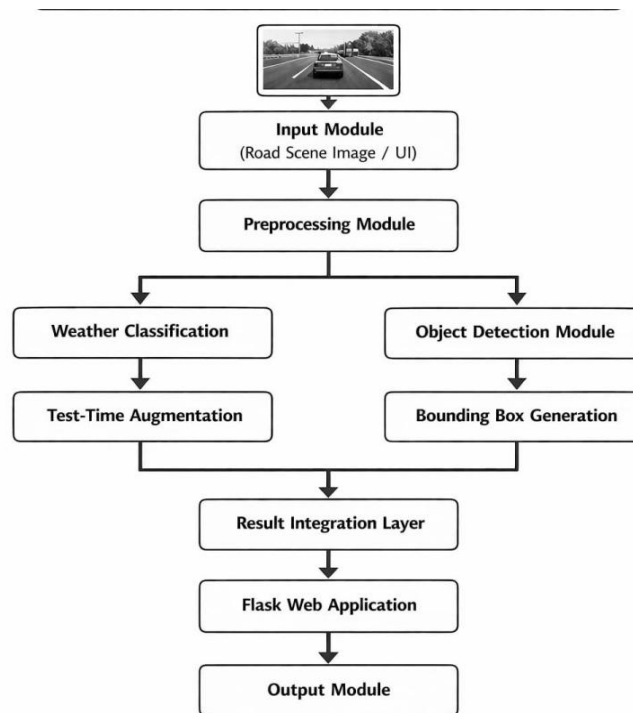


Fig 5.3: System architecture

Model Architecture and Training EfficientNetB0 with ImageNet weights serves as the backbone. A custom classification head consisting of GlobalAveragePooling2D, BatchNormalization, a Dense layer with Dropout, and a softmax output layer is attached. Training proceeds in two phases with ModelCheckpoint, EarlyStopping, and ReduceLROnPlateau callbacks active throughout. The best checkpoint is explicitly loaded after training.

Inference with Test-Time Augmentation At inference, ten augmented versions of each input image are generated and passed through the model. The averaged prediction across all passes produces the final weather class label and confidence scores, improving robustness over a single forward pass.

Object Detection with YOLOv8 YOLOv8n processes the uploaded road scene image and returns bounding box coordinates, confidence scores, and class labels for all detected objects. The annotated image is saved and sent to the frontend along with the weather classification result.

Flask Web Application A Flask server handles image uploads, runs the TTA weather classifier and YOLOv8 detector, and renders results through an HTML frontend. The interface is accessible via any common web browser without requiring any client-side installation or technical expertise.

CHAPTER 6

SYSTEM DESIGN

6.1 MODULE AND ITS DESCRIPTION

The system is composed of seven interconnected parts. Each module oversees a specific stage of the pipeline due to its modular architecture, which for independent testing, maintenance, and possible expansion.

Module1: Dataset Collection and Organisation

This module uses the Kaggle API to obtain the DAWN dataset from Kaggle and classifies it into four weather categories: fog, rain, snow, and sand. For uniformity, folder names are all lowercase. By going through each category's subdirectories, gathering all valid image paths (JPG, PNG, and JPEG), and copying them into the appropriate train and validation directories, an 80/20 random train-validation split is carried out. In order to prevent the identical image from appearing in both sets, the split is carried out at the image level. The training and validation distributions are guaranteed to be representative of each weather class by this module.

Module2: Class-specific augmentation

This module uses the Albumentations library to apply weather-category-specific augmentation to training photos. For each weather class, four separate augmentation pipelines are defined. The fog pipeline uses brightness-contrast variation, Gaussian blur, and RandomFog. The rain pipeline adds moisture cues that visually distinguish rain from snow by using RandomRain with an elevated brightness coefficient (0.85) and HueSaturationValue changes. In order to prevent the over-whitening that happened at 2.5 in the previous pipeline, the snow pipeline uses RandomSnow with a lower brightness coefficient (1.8). The sand pipeline uses GaussNoise, contrast reduction, RGBShift with a high red-channel bias to approximate dusty warmth, and RandomFog at reduced density.

The augmented version of each training image is saved with a 'aug_' prefix after being processed by its class-assigned pipeline. The original training photos and augmented images are then combined into a single final_dataset directory. This fixes the key problem in the original pipeline, which caused

cross-class confusion in the training data by applying augmentations indiscriminately across all classes.

Module3: Data Generator and Preprocessing

For the training and validation sets, this module generates instances of Keras ImageDataGenerator. The crucial adjustment in this case is to substitute `preprocessing_function=preprocess_input` from `tensorflow.keras.applications.efficientnet` for `rescale=1/255`. When an external rescale is used in place of `preprocess_input`, EfficientNetB0's internal normalization layers—which are calibrated to the statistics of its ImageNet training data—are circumvented, resulting in a significant accuracy loss of up to 20–30%. To further diversify the training data, the training generator also performs runtime augmentations (rotation, zoom, horizontal flip, width and height changes). Shuffle is turned off to guarantee uniform evaluation ordering, and the validation generator just uses `preprocess_input` without any geometric augmentation. To account for the class imbalance in the DAWN dataset, class weights are calculated using scikit-learn's `compute_class_weight` with `strategy='balanced'`.

Module 4: Model Architecture and Training

The EfficientNet based weather classifier is defined and trained in this module. ImageNet weights and `include_top=False` are loaded into the underlying EfficientNetB0 model. GlobalAveragePooling2D, BatchNormalization, a 256-unit Dense layer with ReLU activation and 0.4 Dropout, and a softmax output layer with four units corresponding to the four weather classifications make up the custom classification head that is attached.

There are two stages of training. Only the custom head is trained in Phase 1, while the base model is frozen. The loss function is categorical cross-entropy with label smoothing of 0.1, and the optimizer is Adam with a learning rate of 1e-3. Up to 30 epochs of training are conducted. The underlying model is partially unfrozen in Phase 2, with previous layers staying frozen and layers starting at index 100 becoming trainable. Adam is used for fine-tuning, which lasts for up to 20 epochs at a lower learning rate of 1e-4.

During both stages, three callbacks are in use: When validation accuracy surpasses the prior best, ModelCheckpoint saves the model; If validation accuracy does not increase for six consecutive epochs, EarlyStopping stops training; and ReduceLRWhen validation loss plateaus over three

successive epochs with a minimum floor of $1e-7$, OnPlateau halves the learning rate. Prior to evaluation and deployment, the optimal stored checkpoint is explicitly loaded following training.

Module 5: Inference with Test-Time Augmentation

During inference time, this module manages prediction on new images. The system uses a TTA data generator with horizontal flipping, magnification, and brightness variation to produce ten augmented versions of each input image instead of just one forward pass. The unaugmented initial pass and all ten passes' predictions are gathered and averaged. The class with the highest average probability is considered the final prediction, and per-class confidence scores are calculated using the average probability vector. This method makes the system more resilient to changes in camera angle, lighting, or compression artifacts by lowering prediction variance and increasing accuracy on images that deviate from the training distribution.

Module 6: Object Detection with YOLOv8

This module does real-time item recognition on the uploaded road scene image using Ultralytics' YOLOv8n model. A lightweight nano-scale version of YOLOv8 that can be implemented on conventional hardware without a GPU is called YOLOv8n. After processing the image, the model provides bounding box coordinates, confidence scores, and class labels for each object it finds. Before being sent to the frontend, detected class names are deduplicated. Bounding boxes are drawn on an annotated image using the built-in result. The weather categorization result is returned to the user along with the plot() method, which is saved to the static/uploads directory.

Module 7: Flask Web Application and Frontend

The system's deployment layer is provided by this module. HTTP GET and POST requests are handled at the root route by a Flask server (app_fixed.py). After receiving an uploaded image file on POST, the server saves it to static/uploads, does YOLOv8 detection, runs the TTA weather classification pipeline, saves the annotated output image, and uses Jinja2 context variables to return the results to the HTML template. After submitting the styled upload form, the frontend (index.html) shows the annotated image along with the list of detected items and the anticipated weather class. Any common web browser can access the interface, and client-side installation is not necessary.

6.2 INPUT DESIGN

The process of specifying how data is entered into the system to guarantee precision, consistency, and usability is known as input design. An photograph of a road scene submitted by the user using the Flask-based web interface serves as the system's main input in this project. The HTML frontend has an easy-to-use file upload form that supports JPG, JPEG, and PNG image formats. After the user chooses and submits an image, the Flask server receives it and stores it in the static/uploads directory for additional processing. Both the YOLOv8 object detection model and the EfficientNet weather classification pipeline use the uploaded image as their only input.

To ensure proper normalization, the image is preprocessed using EfficientNet's native `preprocess_input` function and shrunk to 224x224 pixels before being sent to the classification model. The raw uploaded image is given straight into YOLOv8n, which takes care of its own preprocessing, for object detection. The system is accessible and useful for real-world use because the input design is kept simple and user-friendly, requiring no technical expertise from the end user.

6.3 OUTPUT DESIGN

The process of establishing how the system's outputs are presented to the user in a way that is understandable, relevant, and accessible is known as output design. In this project, the system generates two main outputs: the object detection result and the weather categorization result, both of which are shown to the user concurrently via the Flask-based HTML frontend. The anticipated weather category, such as fog, rain, snow, or sand, and a confidence score obtained by averaging the predictions of 10 Test-Time Augmentation passes through the EfficientNet model comprise the weather classification output.

An annotated road scene image with bounding boxes created around identified things, such as cars, people, and traffic signs, coupled with the class labels produced by YOLOv8n, make up the object detection output. The user can view the annotated image directly on the web interface when it has been saved to the static/uploads directory. The system is useful and user-friendly for real-world autonomous driving applications since the output is delivered in an easy-to-understand style that doesn't require technical knowledge.

6.4 UML DIAGRAMS

UML (Unified Modeling Language) diagrams are used to represent the system architecture and the flow of data through the system. Three key diagrams are described below.

6.4.1 Use Case Diagram:

The five main use cases of the system—capture image, detect weather, apply weather augmentation, detect road objects, and display results—as well as the interactions between the two main actors—the user and the camera sensor—are depicted in this graphic. By uploading or taking a picture of a road scenario, the user starts the workflow. In response, the system uses YOLOv8 to detect objects, classifies the weather, applies augmentation during training, and provides the user with the annotated output.

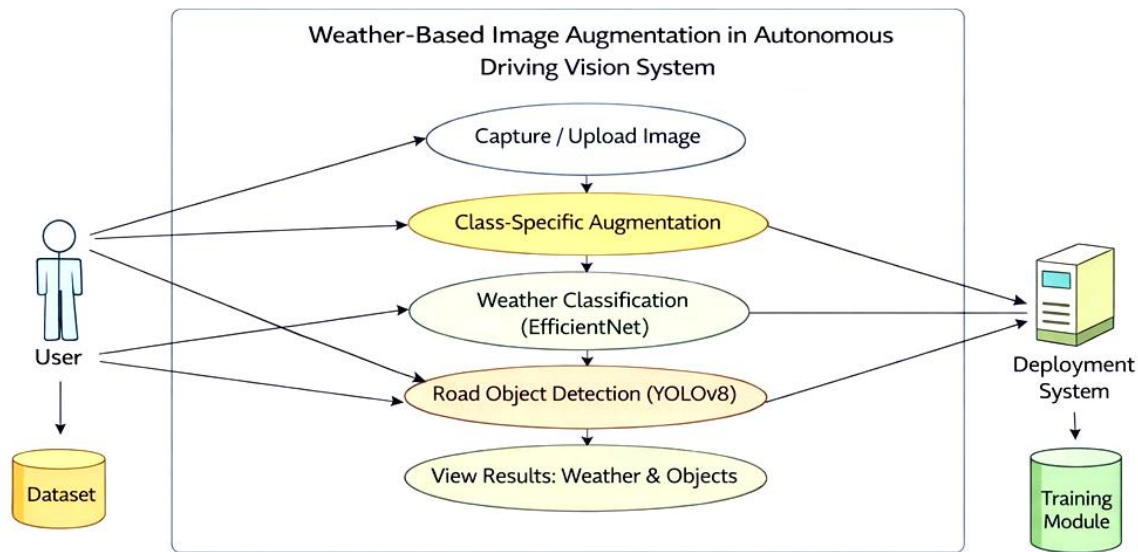


Fig 6.4.1: Use Case Diagram

6.4.2 Class Diagram:

The primary classes and their connections are depicted in this diagram to demonstrate the system's structural design. Important classes include WeatherClassifier (which wraps the EfficientNetB0 model with training, TTA prediction, and checkpoint management methods), ObjectDetector (which

wraps the YOLOv8 model with detection and annotation methods), FlaskApp (which manages web request handling, file I/O, and result rendering), AugmentationPipeline (which encapsulates the four class-specific Albumentations pipelines), and DatasetLoader (which handles dataset downloading, splitting, and directory setup).

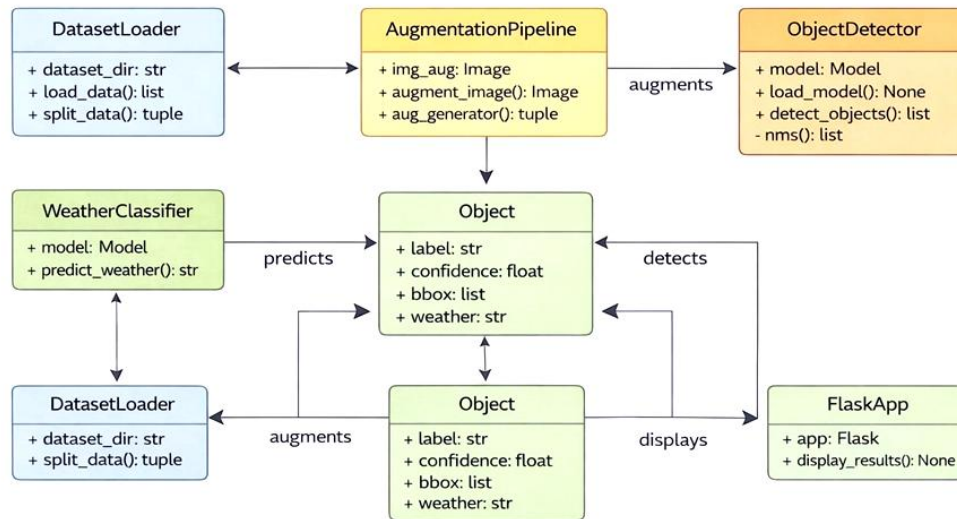


Fig 6.4.2: Class Diagram

6.4.3 Activity Diagram:

The complete workflow from dataset collecting to final result display is depicted in this diagram. Phase 1 training with frozen base, Phase 2 fine-tuning with unfrozen top layers, best model checkpoint loading, TTA inference on new images, YOLOv8 detection, dataset download and organization, train-validation split, class-specific augmentation, merging of original and augmented data, data generator creation with proper preprocessing, class weight computation, and result display via the Flask interface are all included in the order of activities.

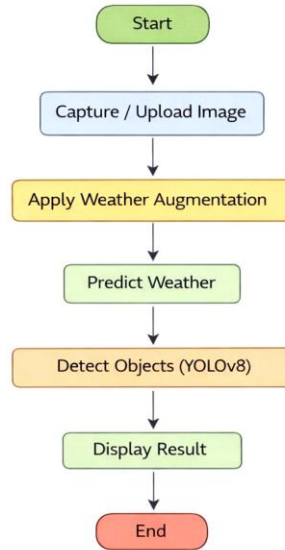


Fig 6.4.3: Activity Diagram

6.4.4 Sequence diagram

The sequence diagram illustrates the dynamic interaction between the key components of the proposed system in a time-ordered sequence. In this project, the diagram captures the flow of communication between the User, Camera, Weather Augmentation module, Weather Detection module, and Object Display module. When the user uploads a road scene image, it is passed through the weather augmentation pipeline, followed by EfficientNetB0-based weather classification and YOLOv8-based object detection. The processed results, including the identified weather condition and detected road objects with bounding boxes, are finally rendered back to the user through the Flask web interface. This diagram confirms that all system components communicate in a well-defined and sequential manner, ensuring accurate and reliable output at every stage of the pipeline.

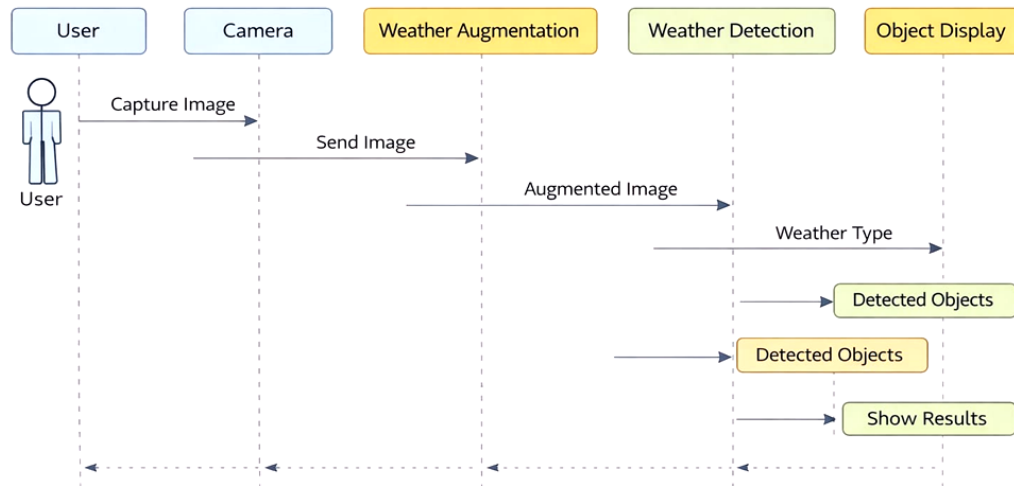


Fig 6.4.4 : sequence diagram

6.4.5 Deployment Diagram

The deployment diagram represents the physical architecture of the proposed system by illustrating how the software components are deployed across hardware nodes. In this project, the deployment diagram captures the distribution of the system across three primary nodes — the Client Machine, the Application Server, and the Deep Learning Model Layer. The user interacts with the system through a web browser on the client machine by uploading road scene images via the Flask-based HTML interface. The Flask server running on the application server receives the uploaded image and coordinates the execution of the EfficientNet weather classification model and the YOLOv8 object detection model, both of which are deployed as trained model artifacts. The processed output, including the predicted weather class and annotated object detection image, is returned to the client browser for display. During the training phase, the model was developed and trained on Google Colab using a T4 GPU, and the final trained model weights were exported and integrated into the Flask deployment environment for CPU-based inference. This deployment architecture ensures that the system is lightweight, portable, and accessible through any standard web browser without requiring any additional client-side installation.

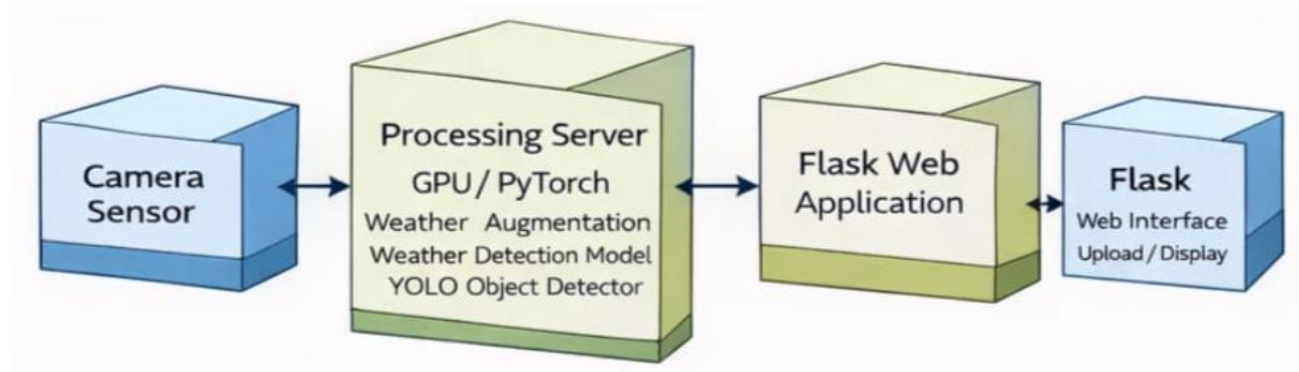


Fig 6.4.5 Deployment diagram

CHAPTER 7

SYSTEM TESTING

System testing is carried out to verify that all modules of the proposed system function correctly both individually and as an integrated whole. The testing phase covers the complete pipeline from dataset preprocessing and augmentation through model training, inference, object detection, and web interface rendering. The primary objective is to confirm that the system produces accurate and reliable results across diverse weather conditions and input image types.

7.1 Types of Testing

7.1.1 Unit Testing:

Before integration, unit testing confirms that each system module functions as intended. The following units are tested separately in this project. The dataset collection module is tested by confirming that the 80/20 split generates the anticipated amount of photos in each class directory, that the DAWN dataset downloads properly, and that folder renaming to lowercase is successful. In order to verify that fog, rain, snow, and sand effects are appropriately applied without distorting the image, the augmentation module is evaluated by running a single image from each weather class through its assigned augmentation pipeline.

The preprocessing module is verified by verifying that the validation generator does not shuffle its output and that `preprocess_input` scales the pixel values to the anticipated range. By contrasting calculated weights with expected values obtained from the training class distribution, the class weight module is evaluated. To ensure that all layers are correctly connected and that the output shape corresponds to the number of classes, the model architecture module is checked by publishing a model summary. To confirm that the averaged prediction is more stable than a single-pass prediction, the TTA inference module is evaluated using a known test image. To verify that bounding boxes and class labels are returned accurately, the YOLOv8 module is tested using photos that contain recognized objects. By launching the server locally and confirming that GET and POST requests yield the anticipated HTML replies, the Flask application module is tested.

7.1.2 Integration Testing:

Integration testing verifies that the modules work together as intended. The following are the main integration points that were examined. The data generator receives the output photos from the augmentation module, and once preprocess_input is applied, the generator's batches are confirmed to have the proper shape, dtype, and value range. The TTA inference module is merged with the trained EfficientNetB0 model, and the averaged prediction output is confirmed to be a legitimate probability distribution that adds up to 1.0. To ensure that the picture upload, TTA prediction, YOLOv8 recognition, image annotation, and template rendering are all appropriately chained in the proper order, the Flask route handler is tested. It is confirmed that the HTML template receives and displays the annotated picture path that YOLOv8 gave.

7.1.3 System Testing:

System testing assesses the entire pipeline as a whole. Road scene photos from all four weather situations are used to test the entire pipeline. Every photograph is uploaded using the Flask web interface, processed by the YOLOv8 detector and TTA weather classifier, and the accuracy of the results is confirmed. System testing verifies that the output image, weather label, and identified objects are consistently and precisely returned, that all modules function flawlessly in sequence, and that the latency is suitable for web-based use.

7.1.4 Performance Testing:

Performance testing is done to evaluate the accuracy and efficacy of the proposed system. It assesses how well the trained object detection model performs in different input image processing circumstances. The purpose of performance testing is to confirm that the system generates reliable and accurate detection results at a suitable speed.

This study evaluates the system's effectiveness using images of a variety of road objects, such as cars, pedestrians, and traffic signs, under a range of weather conditions. The trained model generates detection results with bounding boxes and class labels after examining these images.

The system's performance is evaluated using metrics such as precision, recall, and mean average precision. These measurements help evaluate how well the system handles different weather

conditions and how accurately it detects objects. Performance testing confirms that the proposed system offers reliable object detection and operates with adequate precision.

7.1.5 White Box Testing

White box testing looks at each module's core logic and code structure. The internal behaviors listed below have been confirmed. It is verified that the augmentation loop applies the appropriate pipeline from `aug_map` and iterates independently over each class folder. It is verified that the training loop logs the right callback events at the right epochs, including checkpoint saves, early stopping, and learning rate reductions. Before calculating the mean, the TTA loop is verified to produce precisely 10 predictions for each image (one original and nine enhanced). It has been verified that the checkpoint loading step loads weights from `best_weather_model.keras` instead of training memory. Before sending the identified class names to the template, the YOLO result processing is verified to deduplicate them.

7.1.6 Black Box Testing

Without looking at the internal implementation, black box testing assesses the system based on its observable inputs and outputs. Each weather class is represented by a collection of road scene photos that are uploaded to the Flask web interface. An accurate weather label, a confidence score, a list of identified road items, and an annotated image with bounding boxes are the anticipated results. Expectations and actual outputs are contrasted. Images that clearly show fog should be categorized as such; the same is true for rain, snow, and sand. Images of cars, pedestrians, and traffic signs ought to provide matching detection labels. Correct end-to-end behavior is confirmed by all test inputs producing outputs that match their expected weather and item classifications.

7.2 Test Cases

Test cases are designed to validate each function and module of the system. Here are a few test scenarios:

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
TC01	Verify image upload	Road scene image	Image uploaded successfully	Image uploaded successfully	Pass
TC02	Verify preprocessing module	Raw dataset image	Image resized and normalized correctly	Image processed correctly	Pass
TC03	Verify weather augmentation	Preprocessed image	System generates fog, rain, or snow augmented images	Augmented images generated	Pass
TC04	Verify weather classification	Augmented test image	Correct weather class predicted with confidence score	Correct class predicted	Pass
TC05	Verify object detection	Test image with vehicles and pedestrians	Bounding boxes drawn around detected objects	Objects detected with bounding boxes	Pass
TC06	Verify result display	Detection output	System displays labels and confidence scores on UI	Results displayed correctly	Pass

CHAPTER 8

SYSTEM IMPLEMENTATION

This chapter describes the step-by-step implementation of the proposed system. The implementation is divided into two primary scripts: `MajColabCode.py`, which handles all training-side operations executed in Google Colab, and `app.py`, which provides the Flask-based inference and deployment server. The HTML frontend is served via `index.html`.

DATASET PREPARATION

Setting up the Kaggle API credentials in the Google Colab environment is the first step in the implementation process. Permissions are limited, the `kaggle.json` file is uploaded, and it is stored in the `~/.kaggle` directory. The Kaggle CLI tool is used to download the DAWN dataset, BDD100K dataset (rain), ACDC dataset(fog,rain), which is then extracted into a local `dawn/` directory. To provide naming consistency throughout the pipeline, the extracted subdirectories are renamed from title-case (Fog, Rain, Sand, Snow) to lowercase (fog, rain, sand, snow).

Next, an 80/20 train-validation split is carried out. The implementation makes use of `OS` for every weather type.walk to gather all valid image paths (JPG, PNG, and JPEG) from subdirectories in a recursive manner. The list is divided at the 80% index after being randomly shuffled. The locations of the training and validation pictures are `weather_dataset/train/` and `weather_dataset/val/`, respectively. For verification, the split sizes for each class are printed. By doing this, data leakage between training and validation sets is prevented.

CLASS-SPECIFIC AUGMENTATION

Four separate augmentation pipelines, one for each weather class, are defined using the `Albumentations` library. The fog pipeline consists of `GaussianBlur`, `RandomBrightnessContrast`, and `RandomFog` with a fog coefficient range of 0.3 to 0.7. The rain pipeline simulates the wet-surface saturation cue by combining `RandomRain` with a brightness coefficient of 0.85 (increased from the initial 0.7 to visually distinguish rain from snow), `MotionBlur`, `RandomBrightnessContrast`, and `HueSaturationValue`. The snow pipeline consists of `RandomBrightnessContrast`, `GaussianBlur`, and `RandomSnow` with a brightness coefficient of 1.8 (lowered from the initial 2.5 to avoid over-whitening). To simulate decreased ambient light through dust, the sand pipeline combines

RandomFog at lower density, RGBShift with a significant red channel bias, GaussNoise, GaussianBlur, and RandomBrightnessContrast with a negative brightness bias.

Aug_map, a dictionary keyed by class name, contains all four pipelines. The augmented image is saved to augmented_dataset/train/<class>/ with a 'aug_' prefix after the augmentation loop iterates over each class directory in weather_dataset/train, reads each image with OpenCV, converts it from BGR to RGB, and applies the class-appropriate processing. Next, by copying both source sets into the combined directory, the original and augmented images are combined into final_dataset/train.

DATA GENERATORS AND CLASS WEIGHTS

Instances of Keras ImageDataGenerator are made for validation and training. Preprocessing_function=preprocess_input is imported from TensorFlow for both generators.keras.applications.efficientnet. This correction is crucial: This particular function, which applies the normalization required by the network's internal BatchNormalization layers, is what EfficientNetB0 anticipates processing inputs. Instead, this internal calibration is circumvented by using rescale=1/255, which results in a considerable accuracy loss of typically 20 to 30 percentage points on validation accuracy.

To further diversity training inputs, the training generator also applies rotation (up to 15 degrees), zoom (up to 20%), horizontal flipping, and tiny width and height adjustments. To guarantee a constant evaluation order, the validation generator only uses preprocess_input and disables shuffle. With a batch size of 32, both generators flow from directories at a target size of 224 x 224 pixels.

Sklearn.utils.class_weight is used to calculate class weights.compute_class_weight with strategy='balanced'. This corrects for the inherent imbalance in the DAWN dataset, where some weather situations have substantially more photos than others, by giving greater loss weights to underrepresented classes and lower weights to overrepresented ones.

MODEL ARCHITECTURE

ImageNet weights are loaded when the EfficientNet model is created with `include_top=False` and `input_shape=(224, 224, 3)`. At first, the base model is frozen (`trainable=False`). The output of the base model has a custom classification head added to it: A Dense layer with 256 units and ReLU activation provides the classification representation; a Dropout layer with rate 0.4 regularizes the head to prevent overfitting; a final Dense layer with softmax activation generates the four-class probability distribution; GlobalAveragePooling2D reduces the spatial dimensions to a feature vector; and BatchNormalization normalizes the activations. The Keras functional API is used to build the entire model.

CALLBACKS

Prior to training, three Keras callbacks are defined. The model is saved to `best_weather_model` by `ModelCheckpoint.keras` with `save_best_only=True` anytime validation accuracy above the current best. `EarlyStopping` keeps an eye on validation accuracy and stops training after six consecutive epochs if no improvement is seen. At termination, the best weights are restored. When no improvement is found for three consecutive epochs, `ReduceLROnPlateau` monitors validation loss and lowers the learning rate by a factor of 0.5, with a minimum learning rate floor of $1e-7$. Together, these callbacks provide effective training, constant preservation of the optimal model, and dynamic learning rate adaptation to the loss landscape.

TWO-PHASE TRAINING

The EfficientNetB0 base model is frozen throughout Phase 1, and only the custom classification head is trained. Adam is the optimizer, and his learning rate is $1e-3$. By softening the one-hot targets, the loss is `CategoricalCrossentropy` with `label_smoothing=0.1`, preventing the model from getting overconfident. Class weights are applied during training for a maximum of 30 epochs. There are three active callbacks.

In order to maintain low-level features gained from ImageNet, Phase 2 refines the model by unfreezing layers in the base model starting at index 100. Adam, the optimizer, uses a lower learning rate of $1e-4$ to achieve precise, tiny weight adjustments. The callbacks and loss function are the same. Up to 20 further epochs are used for fine-tuning.

The best checkpoint is specifically loaded from `best_weather_model.keras` using `load_model` following both training stages. This guarantees that the deployed model is unaffected by any transient degradation in the last training epochs.

EVALUATION

The validation set is used to assess the loaded optimal model. `model.assess` prints the final validation loss and accuracy. In order to ensure that the presented metrics accurately reflect the model's performance on a consistent held-out set, this evaluation employs the validation generator with shuffle disabled.

INFERENCE WITH TTA

TTA for single-image inference is implemented via the `predict_weather` function. A 224 x 224 pixel input image is loaded and transformed into a raw NumPy array without scaling. The preprocessing function `preprocess_input`, horizontal flip, 10% zoom, and brightness variation between 0.85 and 1.15 are used to produce a TTA `ImageDataGenerator`. `Preprocess_input` is used to preprocess the original image before it is run through the model for the initial forecast. The TTA generator is iterated to produce nine more augmented versions, which are then run through the model. Each of the ten prediction vectors is averaged by element. The complete per-class probability distribution is printed for analysis, and the class with the highest average probability is used as the final prediction.

FLASK DEPLOYMENT

At server startup, the Flask application loads the YOLOv8n model from `yolov8n.pt` and the trained weather model from `weather_final.keras`. To correspond with the training class index mapping, the class labels—fog, rain, sand, and snow—are hardcoded in alphabetical order. `Static/uploads` is where uploaded photos are stored. By using ten augmented passes and averaging predictions, the `predict_weather_tta` function replicates the TTA inference logic from training.

CONCLUSION AND FEATURE ENHANCEMENT

Conclusion

A weather-aware adaptive vision pipeline for autonomous driving applications has been developed, constructed, and deployed by this project. Through a number of focused technical adjustments and architectural enhancements, the system tackles the main drawbacks of current methods. In order to avoid the cross-class confusion caused by random augmentation pipelines, class-specific augmentation strategies using the Albumentations library guarantee that each weather category—fog, rain, snow, and sand—receives augmentation that matches its own visual character. A crucial accuracy bottleneck found in many published implementations is removed by correcting EfficientNet's preprocessing from the naive `rescale=1/255` to the native `preprocess_input` function. By averaging over 10 augmented passes, Test-Time Augmentation during inference increases prediction stability and strengthens the system's resistance to changes in input image quality, angle, and lighting. With minimal processing cost, YOLOv8n offers real-time road object recognition. The entire pipeline is implemented via a Flask web application that can be accessed through any common browser and doesn't require technical knowledge from the user. Without the need for costly image restoration preprocessing or computationally demanding generative models, the suggested system shows that robust autonomous driving perception under unfavorable weather conditions can be achieved through careful data engineering, principled transfer learning, and lightweight real-time detection.

Future Enhancement

Several directions can be explored to extend and improve the system in future work.

Extended Dataset Diversity Four weather classifications are covered by the DAWN dataset. Additional unfavorable situations like smog, driving at night, severe snowstorms, and mixed-condition sceneries may be included in future work. The real-world generalization of the model can be further enhanced by larger and more geographically diversified datasets.

Generative Augmentation: Albumentations are used in the current system to simulate deterministic weather. In order to create more photorealistic weather-degraded photos, future research can incorporate diffusion-based generative models or GANs, especially for extreme or uncommon atmospheric conditions that are challenging to depict with rule-based augmentation.

Multi-Sensor Fusion: In extremely low visibility situations, camera-only perception has intrinsic limits. When optical cameras alone are unable to offer enough information, the detection reliability can be greatly increased by combining additional sensor modalities like LiDAR, radar, or thermal imaging with visual data.

Weather-Adaptive Object Detection: The existing system carries out object detection and weather categorization in simultaneously. In the future, a weather-specific YOLOv8 model may be chosen using the weather classification output, or detection thresholds could be dynamically adjusted according to the identified environmental condition.

Edge Deployment Optimization: At the moment, the system uses the CPU for inference. The pipeline can be optimized for implementation on embedded automotive hardware with stringent latency and power constraints by model quantization, trimming, and translation to TensorRT or ONNX formats.

Continuous Learning: The system can be expanded in production to facilitate online learning, in which edge devices gather and annotate fresh bad weather samples from actual driving situations on a regular basis. The model is then gradually updated without requiring a complete retraining.

Expanded Object Categories: The COCO class set is used in the current YOLOv8n model. Road-specific item categories like traffic lights, lane markings, and road signs can be detected more accurately in bad weather by fine-tuning YOLOv8 using autonomous driving-specific datasets like nuScenes or KITTI.

REFERENCES

- [1] Wang Z., Zhou P., Zhang S., Du W., Chen X. Enhancing Parking Spot Detection in Extreme Weather Conditions: A Deep Learning Approach With Simulated Weather Augmentation. IEEE Conference on Intelligent Transportation Systems, 2025.
- [2] Marathe A., Ramanan D., Walambe R., Kotecha K. WEDGE: A Multi-Weather Autonomous Driving Dataset Built from Generative Vision-Language Models. IEEE Transactions on Intelligent Transportation Systems, 2023.
- [3] Bayramov E., et al. Weather-Informed Vision Enhancement for Autonomous Object Detection. IEEE Transactions on Intelligent Vehicles, 2025.
- [4] Aloufi N., Alnor A., Basuhail A. Enhancing Autonomous Vehicle Perception in Adverse Weather: A Multi-Objectives Model for Integrated Weather Classification and Object Detection. IEEE Access, 2024.
- [5] Gurbindo U., Brando A., Abella J., König C. Object Detection in Adverse Weather Conditions for Autonomous Vehicles Using Instruct Pix2Pix. IEEE Conference on Computer Vision Applications, 2025.
- [6] N. Kang, J. Li, Y. Zhang, Improved YOLOv9-Based Object Detection for Autonomous Driving in Adverse Weather Conditions, Elsevier Procedia Computer Science, 2024.
- [7] U. Gurbindo, A. Brando, J. Abella, C. König, Object Detection in Adverse Weather Conditions for Autonomous Vehicles Using Instruct Pix2Pix, IEEE Conference on Intelligent Vehicles, 2025.
- [8] D. Weikert, A. Köring, C. Steup, Adverse Weather Benchmarks and Dataset for Object Detection in Autonomous Driving, IEEE Symposium on Computational Intelligence for Energy, Transport and Environmental Sustainability (CIETES), 2025.
- [9] S. Yacoob, M. Ahmed, R. Khan, Enhancing Object Detection Robustness in Adverse Weather Using Domain Adaptation , Elsevier Procedia Computer Science, 2025.
- [10] S. Garg, A. Baghel, A. Agarwal, D. Toshniwal, Snowy Scenes, Clear Detections: Robust Traffic Signal Detection in Adverse Weather Conditions, International Conference on Intelligent Transportation Systems, 2024.

APPENDIX

The complete source code for this project consists of two primary files — the training script executed on Google Colab and the Flask deployment application. The training code handles dataset preparation, class-specific augmentation using Albumentations, data generator construction with EfficientNet preprocessing, dual-output model building using EfficientNetB3, two-phase training with callbacks, and final model evaluation with confusion matrix generation. The deployment code in `app.py` loads the trained weather classification model and YOLOv8 object detector, accepts image uploads through the Flask route, performs Test-Time Augmentation over ten inference passes, applies weather-conditioned YOLO confidence thresholds, validates the input scene using a rule-based filter, and renders the final result to the web interface. The model is saved in the Keras format and the class label mapping is stored as a JSON file for consistent inference. All uploaded and result images are stored in the `static/uploads` directory and served directly through the Flask static file handler.

A. SOURCE CODE

```
from google.colab import files
files.upload() # upload kaggle.json

!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

!kaggle datasets download -d orville/dawn-detection-in-adverse-weather-nature
!unzip -q dawn-detection-in-adverse-weather-nature.zip -d dawn

!ls dawn/DAWN
print("DAWN downloaded.")

# Download BDD100K weather subset
!kaggle datasets download -d marquis03/bdd100k-weather-classification
!unzip -q bdd100k-weather-classification.zip -d bdd100k
!ls bdd100k

!kaggle datasets download -d khalilusmanuk/acdc-clean-dataset-for-training
!unzip -q acdc-clean-dataset-for-training.zip -d acdc
!ls acdc
import os
```

```

weather_counts = {'fog': [], 'rain': [], 'snow': []}

for root, dirs, files in os.walk("acdc"):
    for f in files:
        if f.lower().endswith(('.jpg', '.png', '.jpeg')):
            full_path = os.path.join(root, f)
            for weather in weather_counts:
                if f"/images/{weather}/" in full_path:
                    weather_counts[weather].append(full_path)

for weather, imgs in weather_counts.items():
    print(f"{weather}: {len(imgs)} images")
import os
import shutil
import random

random.seed(42)

base_path = "dawn/DAWN"

os.rename(f"{base_path}/Fog", f"{base_path}/fog")
os.rename(f"{base_path}/Rain", f"{base_path}/rain")
os.rename(f"{base_path}/Sand", f"{base_path}/sand")
os.rename(f"{base_path}/Snow", f"{base_path}/snow")

# Collect ACDC fog images
acdc_fog = []
acdc_rain = []

for root, dirs, files in os.walk("acdc"):
    for f in files:
        if f.lower().endswith(('.jpg', '.png', '.jpeg')):
            full_path = os.path.join(root, f)
            if "/images/fog/" in full_path:
                acdc_fog.append(full_path)
            elif "/images/rain/" in full_path:
                acdc_rain.append(full_path)

# Copy all ACDC fog → dawn/DAWN/fog
print(f"Copying {len(acdc_fog)} ACDC fog images...")

```

```

for img_path in acdc_fog:
    fname = os.path.basename(img_path)
    shutil.copy(img_path, os.path.join(base_path, "fog", f"acdc_{fname}"))

# Copy all ACDC rain → dawn/DAWN/rain
print(f"Copying {len(acdc_rain)} ACDC rain images...")
for img_path in acdc_rain:
    fname = os.path.basename(img_path)
    shutil.copy(img_path, os.path.join(base_path, "rain", f"acdc_{fname}"))

# Also add BDD100K rainy (500 images)
bdd_rainy_dir = "bdd100k/train/rainy"
rainy_imgs = os.listdir(bdd_rainy_dir)
random.shuffle(rainy_imgs)
rainy_subset = rainy_imgs[:500]
print(f"Copying {len(rainy_subset)} BDD100K rainy images...")
print(f"{cls}: train={len(train_imgs)}, val={len(val_imgs)}")
import albumentations as A
import cv2
import os
import shutil
import random
import numpy as np

# Fog: bright, uniform, soft haze
fog_aug = A.Compose([
    A.RandomFog(fog_coef_range=(0.4, 0.8), alpha_coef=0.1, p=1.0),
    A.RandomBrightnessContrast(brightness_limit=(0.1, 0.3), contrast_limit=(-0.3, -0.1), p=0.9),
    A.GaussianBlur(blur_limit=(5, 9), p=0.8),
    A.RandomGamma(gamma_limit=(90, 130), p=0.5),
    A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=(-40, -20), val_shift_limit=(10, 30),
p=0.7),
])

# Rain: dark, directional, blue-grey wet shift
rain_aug = A.Compose([
    A.RandomRain(slant_range=(-15, 15), drop_length=25, drop_width=1, blur_value=4,
brightness_coefficient=0.75, p=1.0),
    A.MotionBlur(blur_limit=(5, 9), p=0.8),
    A.RandomBrightnessContrast(brightness_limit=(-0.3, -0.1), contrast_limit=(-0.1, 0.1), p=0.8),
    A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=(-20, 10), val_shift_limit=(-20, -5),
p=0.7),
])

```

```

    A.RGBShift(r_shift_limit=(-10, 0), g_shift_limit=(-5, 5), b_shift_limit=(5, 20), p=0.7),
])

# Snow: bright white, low saturation
snow_aug = A.Compose([
    A.RandomSnow(brightness_coeff=1.8, p=0.9),
    A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.05, p=0.5),
    A.GaussianBlur(blur_limit=(3, 5), p=0.4),
    A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=-20, val_shift_limit=15, p=0.4),
])

# Sand: orange-brown tint, dusty haze
sand_aug = A.Compose([
    A.RandomFog(fog_coef_range=(0.2, 0.5), p=0.6),
    A.RGBShift(r_shift_limit=25, g_shift_limit=12, b_shift_limit=5, p=0.8),
    A.GaussNoise(std_range=(0.02, 0.08), p=0.6),
    A.GaussianBlur(blur_limit=(3, 7), p=0.6),
    A.RandomBrightnessContrast(brightness_limit=(-0.2, 0.1), contrast_limit=(-0.3, 0.1), p=0.6),
    cv2.imwrite(
        os.path.join(out_dir, f"aug_{count}_{img_name}"),
        cv2.cvtColor(augmented, cv2.COLOR_RGB2BGR)
    )
    count += 1

total = len(os.listdir(out_dir))
print(f"{cls}: {total} images (target={target})")

```

app.py

```

from flask import Flask, request, render_template
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.efficientnet import preprocess_input
from ultralytics import YOLO

import numpy as np
import os
import cv2
import json
import tensorflow as tf

```

```

app = Flask(__name__)

weather_model = load_model("weather_final.keras")
yolo_model = YOLO("yolov8s.pt")

# — Class labels —————
with open("class_indices.json") as f:
    class_indices = json.load(f)
classes = [k for k, v in sorted(class_indices.items(), key=lambda x: x[1])]
print("Classes:", classes)

# — Road-relevant YOLO object classes —————
PRIMARY_ROAD_CLASSES = {
    'car', 'truck', 'bus', 'motorcycle', 'bicycle',
    'traffic light', 'stop sign', 'train', 'van'
}

SECONDARY_ROAD_CLASSES = {'person'}

# — Weather-specific YOLO confidence thresholds —————
WEATHER_CONF = {
    'fog': 0.25,
    'rain': 0.30,
    'sand': 0.25,
    'snow': 0.35,
}

# — Validity thresholds —————
WEATHER_CONF_THRESHOLD = 0.60
MIN_TOP2_GAP = 0.15

```

```
UPLOAD_FOLDER = "static/uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# — TTA prediction —————
def predict_weather_tta(filepath, n_augments=10):
    img = image.load_img(filepath, target_size=(300, 300))
    img_array = image.img_to_array(img)

    tta_datagen = tf.keras.preprocessing.image.ImageDataGenerator(
        preprocessing_function=preprocess_input,
```

B. SCREEN SHOTS

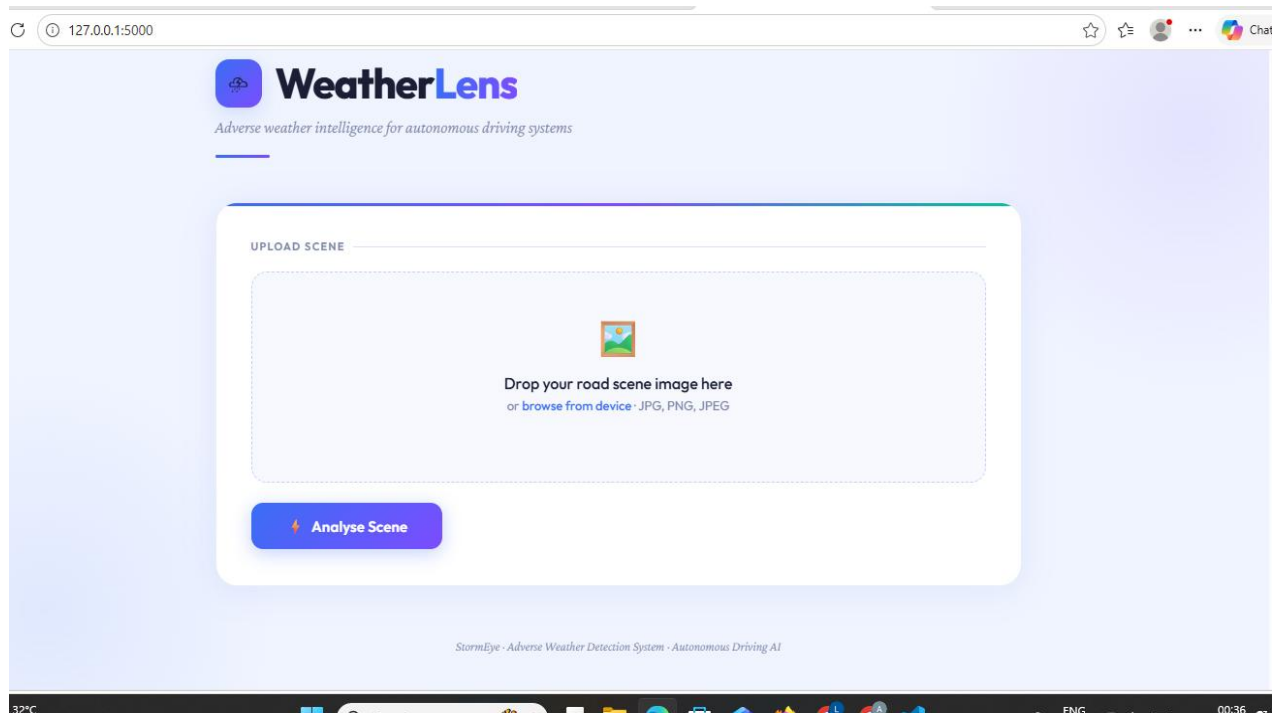


Fig B.a: User Interface

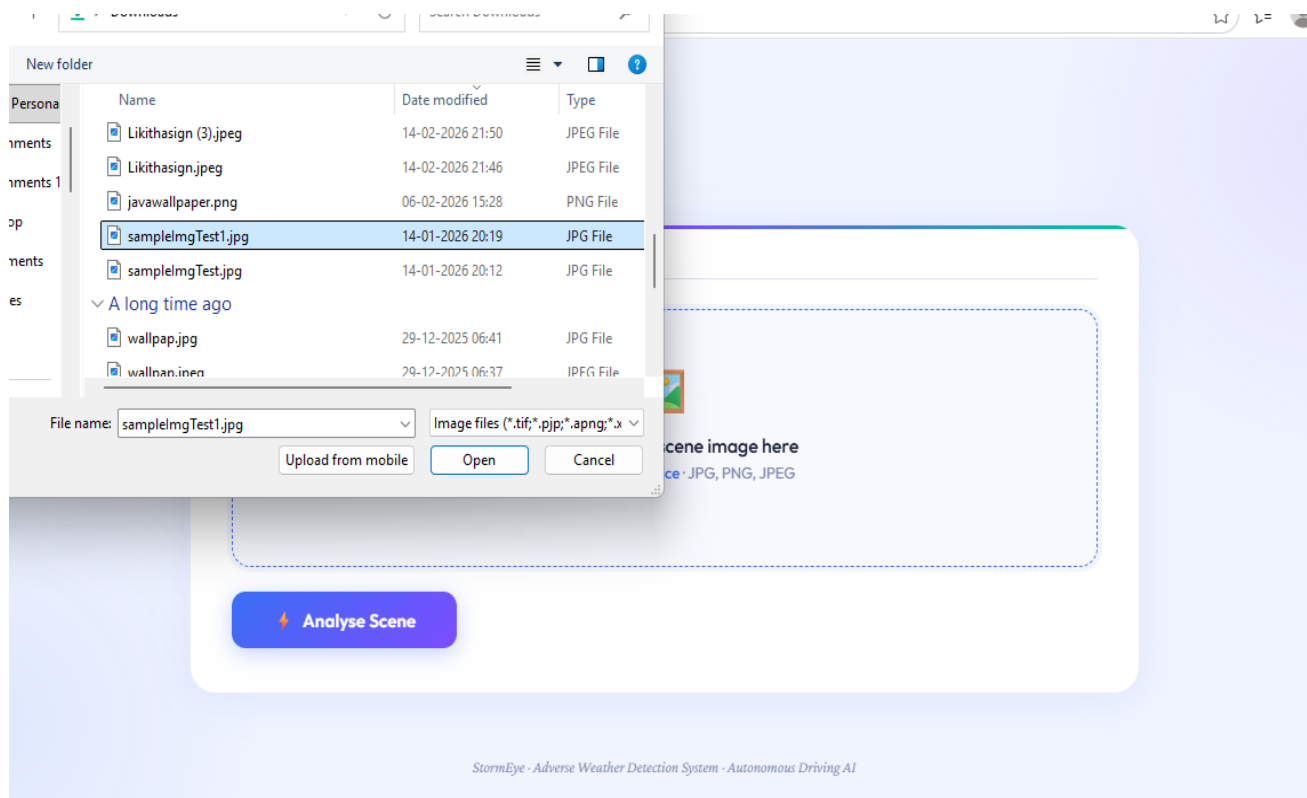


Fig B.b : Uploading Image

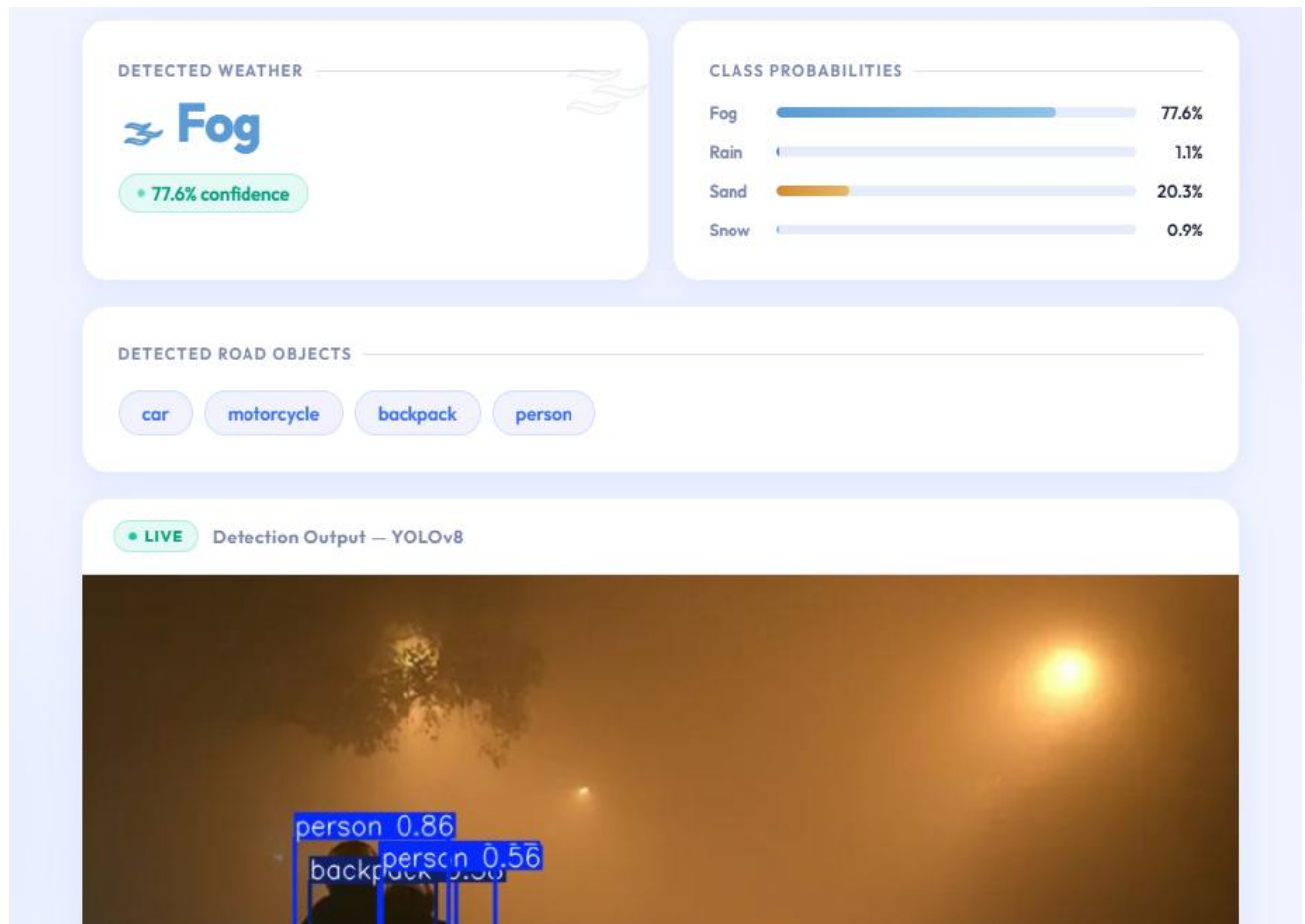


Fig B.c : Dectected Weather & Class Confidences

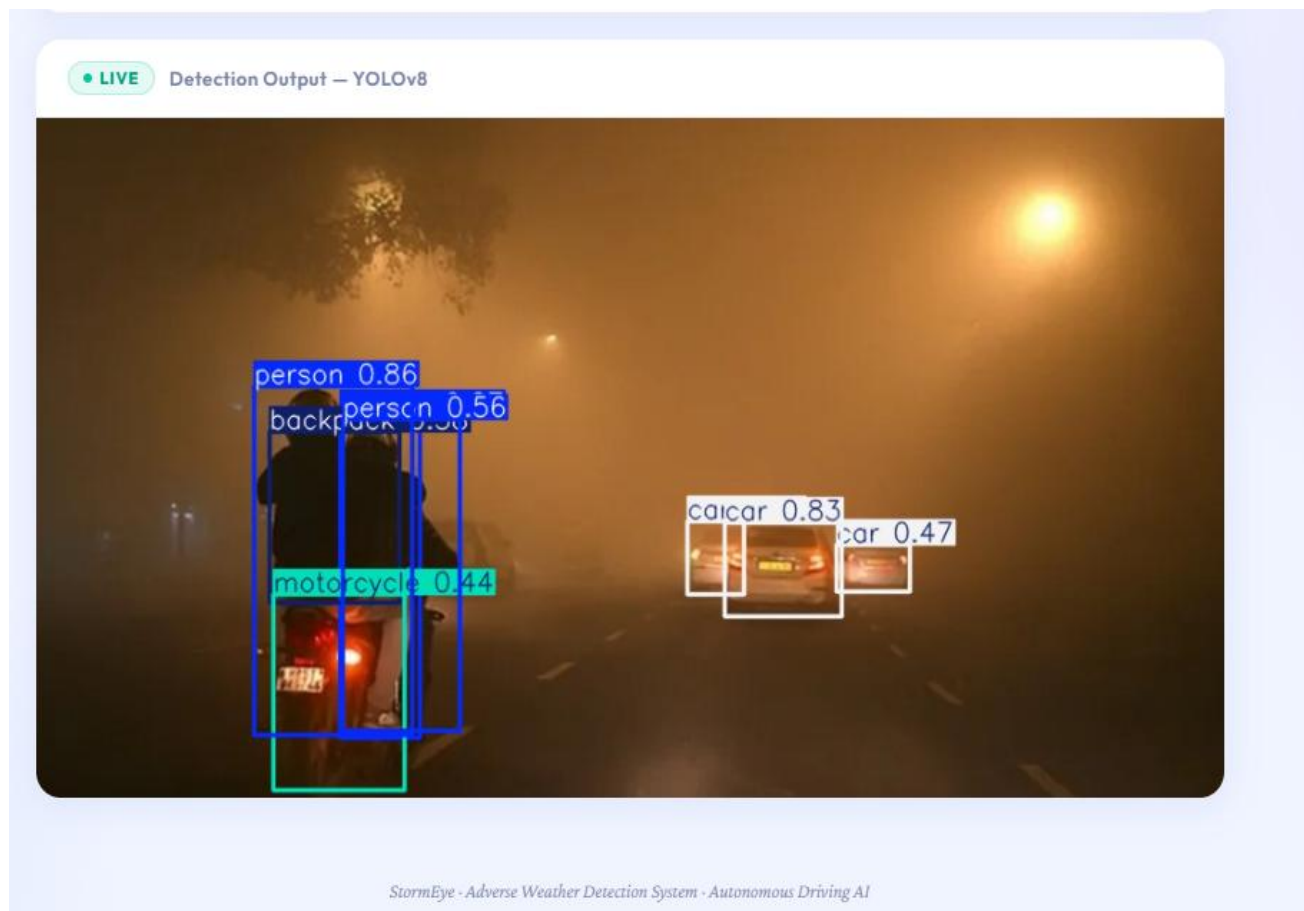


Fig B.d : Object detection on Image



Invalid input — please upload a road or weather scene image.

Please upload a road or driving scene image for accurate detection.



Fig B.e: Invalid Image Upload

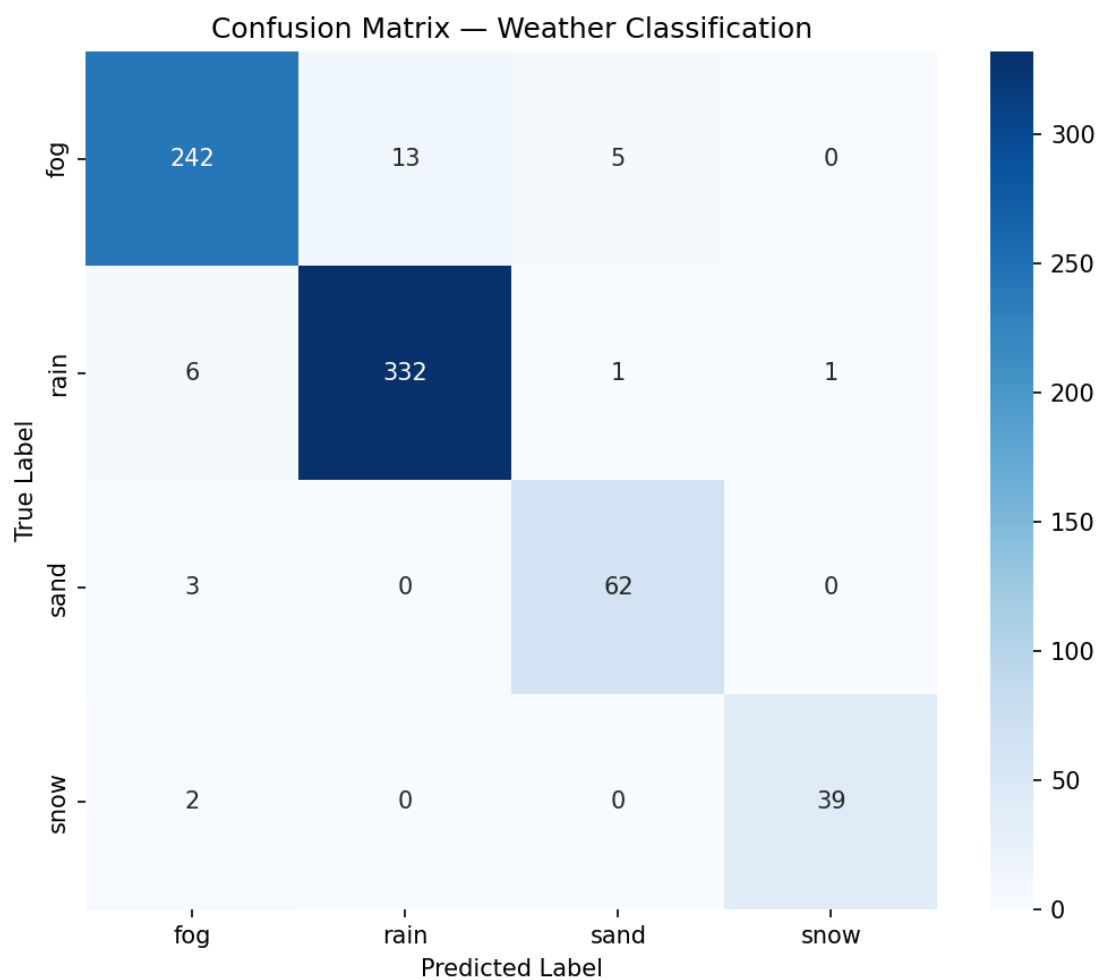


Fig B.f : Confusion matrix

ANNEXURE I

Title of the Project	Weather-Based Image Augmentation for enhancing object detection in Autonomous Driving Vision System
Name of the Student	H Likitha – 22751A3120
	G Harshitha – 22751A3116
	P Kavya – 22751A3143
Name of the Guide & Designation	Mr. L. Vijaya Kumar, Assistant Professor

Table 1: Outcomes attained and its Justification

PO	Justification
PO1	Gained basic knowledge on the project area while selecting the problem.
PO2	Learned how to analyze the complex problems.
PO3	Learned problem solving skills while implementing modules.
PO4	Learned how to conduct investigations on complex real-world problems.
PO5	Learned modern tool usage while developing source code.
PO6	Learned professional engineering practices while adding new features to the project.
PO7	Learned how this project impacts on societal and environmental contexts.
PO8	Learned software ethics to be followed while developing this project.
PO9	Learned how to work as an individual and as a team member during this entire project period.
PO10	Learned how to communicate effectively during the project reviews.
PO11	Learned how to estimate cost benefit analysis for the project management.
PO12	The knowledge which I have gained through the project will be helpful for my future to refine my skills.

A Plagiarism Check Report
On
WEATHER-BASED IMAGE AUGMENTATION FOR ENHANCING
OBJECT DETECTION IN AUTONOMOUS DRIVING VISION
SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

CSE (ARTIFICIAL INTELLIGENCE)

By

H LIKITHA

22751A3120

G HARSHITHA

22751A3116

P KAVYA

22751A3143

Under the guidance of

Mr. L. VIJAYA KUMAR,

Assistant Professor

CAI_Batch6.pdf

 Vydehi Institute of Medical Sciences & Research Centre

Document Details

Submission ID

trn:oid:::3618:134417000

Submission Date

Apr 7, 2026, 10:28 AM GMT+5:30

Download Date

Apr 7, 2026, 10:30 AM GMT+5:30

File Name

CAI_Batch6.pdf

File Size

1.6 MB

69 Pages

15,436 Words

93,774 Characters

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **163 Not Cited or Quoted 11%**
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 7%  Internet sources
- 8%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Replaced Characters**
55 suspect characters on 22 pages
Letters are swapped with similar characters from another alphabet.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **163 Not Cited or Quoted 11%**
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 7%  Internet sources
- 8%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** **Publication**
Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%
- 2** **Internet**
www.mdpi.com <1%
- 3** **Internet**
www-dev.fit.vutbr.cz <1%
- 4** **Internet**
zir.nsk.hr <1%
- 5** **Publication**
K. V. Sambasivarao, Anasuya Sesha Roopa Devi Bhima. "Artificial Intelligence, Co... <1%
- 6** **Internet**
arxiv.org <1%
- 7** **Publication**
Xinyun Feng, Tao Peng, Ningguo Qiao, Haitao Li, Qiang Chen, Rui Zhang, Tingting... <1%
- 8** **Internet**
vbn.aau.dk <1%
- 9** **Internet**
download.bibis.ir <1%
- 10** **Internet**
www.coursehero.com <1%

11	Internet	alumentations.ai	<1%
12	Publication	Pardhu Thottempudi, Asral Bin Bahari Jambek, Vijay Kumar, Biswaranjan Acharya...	<1%
13	Internet	vtechworks.lib.vt.edu	<1%
14	Publication	S.P. Jani, M. Adam Khan. "Applications of AI in Smart Technologies and Manufactu...	<1%
15	Internet	mau.diva-portal.org	<1%
16	Publication	"Soft Computing and Signal Processing", Springer Science and Business Media LL...	<1%
17	Internet	cam.tju.edu.cn	<1%
18	Internet	www.edgeai-trust.eu	<1%
19	Internet	www.diffchecker.com	<1%
20	Internet	33rdsquare.com	<1%
21	Publication	Pradeep Singh, Balasubramanian Raman. "Deep Learning Through the Prism of T...	<1%
22	Publication	Alok Kumar Tiwari, Manisha Pattanaik, G. K. Sharma. "Low-light DEtection TRansf...	<1%
23	Internet	ebin.pub	<1%
24	Internet	learnopencv.com	<1%

25	Internet	pure.tue.nl	<1%
26	Internet	techiefreak.org	<1%
27	Publication	"Proceedings of the International Conference on Computer Science, Electronics a...	<1%
28	Internet	cm.e-ale.org	<1%
29	Internet	dataplatform.cloud.ibm.com	<1%
30	Internet	zenn.dev	<1%
31	Publication	Deepika Varshney, Preeti Nagrath, Srishti Vashishtha, Victor Hugo C. de Albuquer...	<1%
32	Publication	Yan Gao, Qing He. "Chapter 6 CAD Automatic Optimization Plugin", Springer Scie...	<1%
33	Internet	filecr.com	<1%
34	Publication	Md Zia Uddin. "Trustworthy Multimodal Intelligent Systems for Independent Livi...	<1%
35	Internet	dokumen.pub	<1%
36	Internet	www.primescholars.com	<1%
37	Internet	assets-eu.researchsquare.com	<1%
38	Internet	summit.sfu.ca	<1%

39	Internet	workopportunities.org	<1%
40	Publication	"Ultra-Widefield Fundus Imaging for Diabetic Retinopathy", Springer Science and ...	<1%
41	Publication	Bal S. Virdee, Tanweer Ali, Jaume Anguera, Suman Lata Tripathy. "Connecting Int...	<1%
42	Publication	Shaik Yacoob, D. Phani Kumar, G.G.S. Harshitha, V.Y. Sai Teja, C.H. Satya Veni, K. ...	<1%
43	Publication	Sunil Kumar. "Python for Accounting and Finance", Springer Science and Business...	<1%
44	Internet	link.springer.com	<1%
45	Internet	www.irjmets.com	<1%
46	Publication	Cong Xu, Ravi Sankar. "A Comprehensive Review of Autonomous Driving Algorith...	<1%
47	Publication	Debasis Kumar, Naveed Muhammad. "Object Detection in Adverse Weather for A...	<1%
48	Publication	Nanor Ebenezer, Cobbinah B. Mawuli, Qinli Yang, Junming Shao, Kobiah Christian...	<1%
49	Internet	www.cnblogs.com	<1%
50	Internet	ijisrt.com	<1%
51	Internet	mdpi-res.com	<1%
52	Internet	superml.dev	<1%



**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES
(AUTONOMOUS)**

Accredited by NBA (EEE and MCA)
MURUKAMBATTU, CHITTOOR - 517127, A.P.



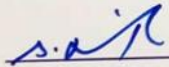
5th INTERNATIONAL CONFERENCE ON
ADVANCES AND INNOVATION IN ENGINEERING AND SCIENCES
2nd APRIL - 2026

**CERTIFICATE
OF APPRECIATION**

This certificate is proudly presented to

MR. / MS. H. Likitha, CSE-A1

has participated and presented a paper titled Weather-Based Image Argumentation
for enhancing object Detection in Autonomous Driving using System
and it has been awarded as the **BEST PAPER** at the 5th INTERNATIONAL CONFERENCE ON
ADVANCES AND INNOVATION IN ENGINEERING AND SCIENCES (ICAIES-2K26) held on 2nd
APRIL 2026.


CONVENOR


PRINCIPAL





**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES
(AUTONOMOUS)**

Accredited by NBA (EEE and MCA)
MURUKAMBATTU, CHITTOOR - 517127, A.P.



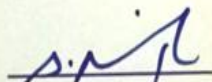
5th INTERNATIONAL CONFERENCE ON
ADVANCES AND INNOVATION IN ENGINEERING AND SCIENCES
2nd APRIL - 2026

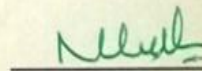
**CERTIFICATE
OF APPRECIATION**

This certificate is proudly presented to

Mr. / Ms. G. Harshitha - CSE-A1

has participated and presented a paper titled Weather - Based Image Argumentation
for Enhancing object Detection in Autonomous Driving using system
and it has been awarded as the **BEST PAPER** at the 5th INTERNATIONAL CONFERENCE ON
ADVANCES AND INNOVATION IN ENGINEERING AND SCIENCES (ICAIES-2K26) held on 2nd
APRIL 2026.


CONVENOR


PRINCIPAL





**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES
(AUTONOMOUS)**

Accredited by NBA (EEE and MCA)
MURUKAMBATTU, CHITTOOR - 517127, A.P.



5th INTERNATIONAL CONFERENCE ON
ADVANCES AND INNOVATION IN ENGINEERING AND SCIENCES
2nd APRIL - 2026

**CERTIFICATE
OF APPRECIATION**

This certificate is proudly presented to

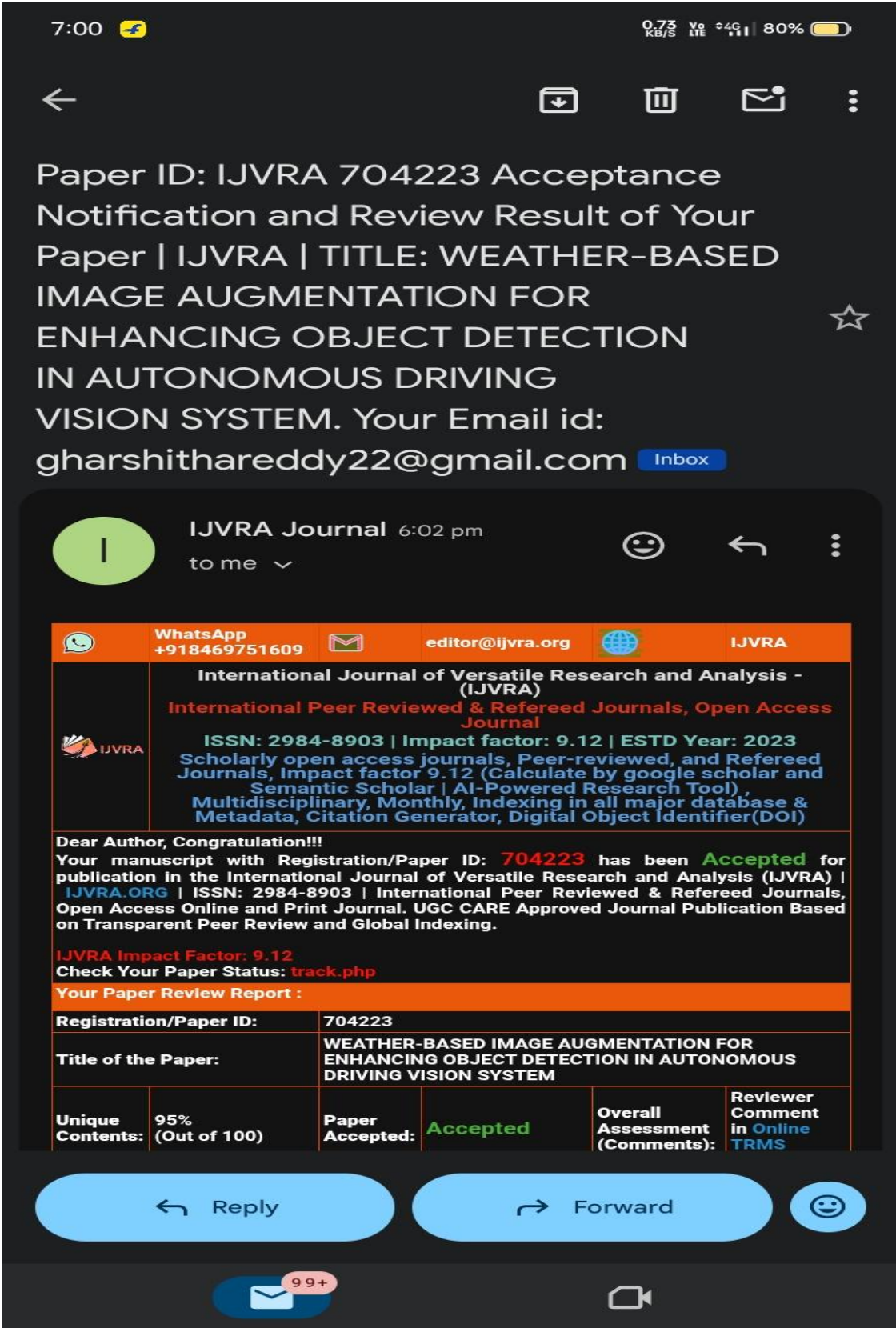
MR. / MS. P. Kavya, CSE-A1

has participated and presented a paper titled Weather-Based Image Argumentation
for enhancing object Detection in Autonomous Driving using system
and it has been awarded as the **BEST PAPER** at the 5th INTERNATIONAL CONFERENCE ON
ADVANCES AND INNOVATION IN ENGINEERING AND SCIENCES (ICAIES-2K26) held on 2nd
APRIL 2026.


CONVENOR


PRINCIPAL







Check Your Paper Status: [track.php](https://ijpub.org/ijvra/track.php)

Your Paper Review Report :

Registration/Paper ID:		704223			
Title of the Paper:		WEATHER-BASED IMAGE AUGMENTATION FOR ENHANCING OBJECT DETECTION IN AUTONOMOUS DRIVING VISION SYSTEM			
Unique Contents:	95% (Out of 100)	Paper Accepted:	Accepted	Overall Assessment (Comments):	Reviewer Comment in Online TRMS System
Publication of Paper:		Paper Accepted. Please complete payment and documents process. Paper will be published Within 01-02 Days after submission of payment proof and documents to editor@ijvra.org .			

Publication/Article Processing Fees

Indian Author	Foreign/International Author
₹ 599 INR	\$ 40 USD

STEP 1: Pay Publication Fees for Indian Author

Preferred Fast Payment Link 1:	https://ijpub.org/ijvra/payfeesonline.php https://ijpub.org/ijvra/payupi.php
--------------------------------	--

(Use Preferred Fast Payment Link 1 if not working then use below options. Using Payment Link 1 paper will be published within 1 to 2 days.)

Other Payment Link 2:	https://ijpub.org/ijvra/indianpaypubcharge.php (Take More time in Payment Verification.)
UPI Payment Link 3:	https://ijpub.org/ijvra/upi.php (Take More time in Payment Verification.)
Payment Link 4:	https://ijpub.org/ijvra/irazorpay.php (Take More time in Payment Verification.)

Only For Foreign/International Author Payment: Kindly Visit this Link

STEP 1: Pay Publication Fees for Foreign/International Author

Payment Link 1:	https://ijpub.org/ijvra/paypalpayment.php
Payment Link 2:	https://ijpub.org/ijvra/fpayfeesonline.php
Payment Link 3:	https://ijpub.org/ijvra/frazorpay.php
Payment Link 4:	https://ijpub.org/ijvra/foreignpaypubcharge.php

Only For Indian Author Payment: Kindly Visit this Link

STEP 2: Send Payment Proof and Documents to editor@ijvra.org

After Publication fees paid complete below process after that publication will take 1 to 2 days. Send payment screenshot and Documents to editor@ijvra.org

To Track Your Paper <https://ijpub.org/ijvra/track.php>

Fill up Forms and Prepare Following Documents:

1. Final Formatted Manuscript/Research paper AS per IJVRA Paper Format (must be in DOC file format and file name should be your registration ID). ([Download](#))
2. Undertaking by Author Form (Scan copy/Screen shot)(Handwritten signed by author) ([Download](#))
3. Identity Proof of corresponding author only (Scan copy/Screen shot) (election Card/PAN Card/College Identity card/Driving License)
4. Payment Proof (Scan copy/Screen shot) also mentions Paid amount, Transaction ID, and Date of Payment in the mail whenever you send these documents.

Note: If all author is not available than complete signature and identity proof process with single author. All author signature and identity proof not compulsory only one author signature and identity proof required.
Please mention your Registration ID in the subject of mail whenever you communicate with us regarding any inquiry related to your research paper publication.

Thank you very much for submitting your article to the IJVRA.

← Reply

→ Forward

